

2.1 引言

在数字电子学中，*circuit*是一种处理离散值变量的网络。电路可视为一个黑箱，如图2.1所示，其

一个或多个离散值一个或多个离散值output terminals>一个描述输入与输出之间关系的functional specification>一个描述输入变化与输出响应之间延迟的timing specification

窥探黑箱内部，电路由节点和元件组成。*element*本身是一个具有输入、输出和规格的电路。*node*是一根导线，其电压传递离散值变量。节点分为input、output或internal。输入从外部世界接收值。输出将值传递到外部世界。既非输入也非输出的导线称为内部节点。图2.2

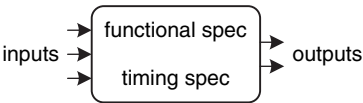


图2.1 作为黑盒的电路，包含输入、输出和规格说明

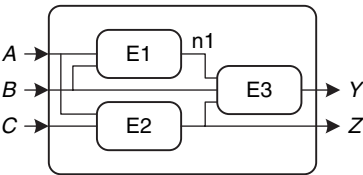


图2.2 单元与节点

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

图示为一个包含三个元件（E1、E2、E3）和六个节点的电路。节点A、B和C为输入，Y和Z为输出。n1是位于E1与E3之间的内部节点。

数字电路分为combinational和sequential两类。组合电路的输出仅取决于输入的当前值；换言之，它通过组合当前输入值来计算输出。例如，逻辑门就是一种组合电路。时序电路的输出则同时取决于输入的当前值和历史值；也就是说，它依赖于输入序列。组合电路是memoryless，而时序电路具有memory。本章重点讨论组合电路，第3章将研究时序电路。

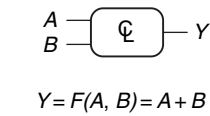


图2.3 组合逻辑电路

组合电路的功能规范根据当前输入值表达输出值。组合电路的时序规范包括从输入到输出延迟的下限和上限。我们首先专注于功能规范，然后在本章后面再回到时序规范。

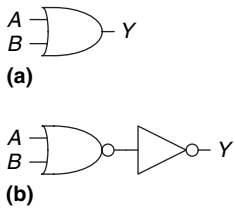


图2.4 两种OR实现

图2.3展示了一个具有两个输入和一个输出的组合电路。图的左侧是输入A和B，右侧是输出Y。框内的符号表示该电路仅使用组合逻辑实现。在此示例中，函数F被指定为或运算： $Y = F(A, B) = A + B$ 。用语言描述，输出Y是两个输入A和B的函数——即 $Y = A$ 或 B 。

图2.4展示了图2.3中组合逻辑电路的两种可能的implementations。正如我们将在本书中反复看到的，单一功能通常有多种实现方式。你需要根据可用的构建模块和设计约束来选择使用哪一种。这些约束通常包括面积、速度、功耗和设计时间。

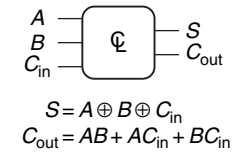


图2.5 多输出组合电路

图2.5展示了一个具有多个输出的组合电路。这个特定的组合电路被称为full adder，我们将在第5.2.1节中再次讨论它。这两个方程根据输入A、B和Cin，指定了输出S和Cout的功能。

为简化绘图，我们常用一条带斜线的单线并在其旁标注数字来表示bus，即多路信号的总线。数字指定了总线中包含的信号数量。例如，图2.6(a)表示一个具有三个输入和两个输出的组合逻辑模块。如果位数不重要或从上下文中显而易见，斜线可以不带数字显示。图2.6(b)表示两个组合逻辑模块，其中一个模块的任意数量输出作为第二个模块的输入。

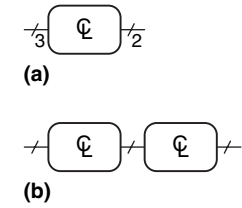


图2.6 多信号的斜杠表示法

combinational composition的规则告诉我们如何从较小的组合电路元件构建大型组合电路。

如果电路由互连的电路元件组成，且满足以下条件，则该电路是组合逻辑电路：

每个电路元件本身都是组合逻辑的。► 电路的每个节点要么被指定为电路的输入，要么恰好连接到一个电路元件的输出端。电路不包含循环路径：通过电路的每条路径最多访问每个电路节点一次。

组合组合的规则是充分的，但并非严格必要。某些违反这些规则的电路仍然是组合电路，只要输出仅取决于输入的当前值。然而，判断这些异常电路是否为组合电路更为困难，因此我们通常将组合组合作为构建组合电路的一种方式。

示例2.1 组合电路

根据组合逻辑电路的组合规则，图2.7中的哪些电路是组合逻辑电路？

解决方案 电路(a)是组合逻辑电路。它由两个组合逻辑电路元件（反相器I1和I2）构成，包含三个节点：n1、n2和n3。n1是电路的输入，也是I1的输入；n2是内部节点，为I1的输出和I2的输入；n3是电路的输出，也是I2的输出。电路(b)不是组合逻辑电路，因为存在一条循环路径：XOR的输出反馈到其一个输入端。因此，从n4开始的循环路径经过XOR到达n5，再返回n4。电路(c)是组合逻辑电路。电路(d)不是组合逻辑电路，因为节点n6同时连接到I3和I4的输出端。电路(e)是组合逻辑电路，展示了两个组合逻辑电路连接形成更大的组合逻辑电路。电路(f)不遵循组合逻辑电路的组合规则，因为它通过两个元件存在一条循环路径。根据元件的功能，它可能是也可能不是组合逻辑电路。

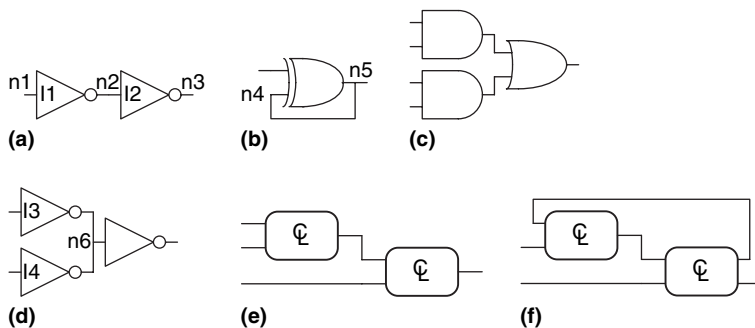


图2.7 示例电路

像微处理器这样的大型电路可能非常复杂，因此我们运用第1章中的原理来管理这种复杂性。将电路视为具有明确接口和功能的黑箱，是抽象与模块化原则的应用。用更小的电路元件构建电路，则是层次化原则的体现。组合电路的构成规则，正是规范性原则的应用。

组合电路的功能规范通常以真值表或布尔方程的形式表示。在接下来的章节中，我们将描述如何从任意真值表推导出布尔方程，以及如何使用布尔代数和卡诺图简化方程。我们将展示如何使用逻辑门实现这些方程，并分析这些电路的速度。

2.2 布尔方程

布尔方程处理的是取值为真或假的变量，因此非常适合描述数字逻辑。本节将定义布尔方程中常用的一些术语，然后展示如何根据真值表为任意逻辑函数编写布尔方程。

2.2.1 术语

变量*complement*的A是其反相A。变量或其补形式称为*literal*。例如，A、A、B和B都是文字。我们将A称为变量的*true form*，将A称为*complementary form*；“真形式”并不意味着A为真，而仅表示A上方没有横线。

一个或多个文字变量的与运算称为*product*或*implicant*。AB、ABC和AB都是三变量函数的蕴含项。*minterm*是包含函数所有输入的一个乘积项。ABC是三变量A、B和C的函数的一个最小项，但AB不是，因为它不包含C。类似地，一个或多个文字变量的或运算称为*sum*。*maxterm*是包含函数所有输入的一个和项。A + B + C是三变量A、B和C的函数的一个最大项。

在解释布尔方程时，*order of operations* 很重要。Y = A + BC 是指 Y = (A 或 B) 与 C，还是 Y = A 或 (B 与 C)？在布尔方程中，NOT 具有最高的 *precedence*，其次是 AND，然后是 OR。就像在普通方程中一样，乘积在求和之前执行。因此，该方程被解读为 Y = A 或 (B 与 C)。方程 2.1 给出了运算顺序的另一个示例。

$$\overline{A}B + BC\overline{D} = ((\overline{A})B) + (BC(\overline{D}))$$

(2.1)

A	B	Y	minterm	minterm name
0	0	0	$\overline{A} \overline{B}$	m_0
0	1	1	$\overline{A} B$	m_1
1	0	0	$A \overline{B}$	m_2
1	1	0	$A B$	m_3

图2.8 真值表与最小项

2.2.2 和之积形式

N个输入的真值表包含2^N行，每行对应输入的一种可能取值。真值表中的每一行都与该行为真值的最小项相关联。图2.8展示了一个包含两个输入A和B的真值表，每行都显示了其对应的最小项。例如，第一行的最小项是AB，因为当A = 0、B = 0时AB为真。最小项从0开始编号：第一行对应最小项0，即m₀；下一行对应最小项1，即m₁，以此类推。

我们可以通过将输出Y为TRUE的每个最小项相加，为任何真值表写出布尔方程。例如，在图2.8中，只有一行（或最小项）的输出Y为TRUE，用蓝色圆圈标出。因此， $Y = \overline{A}B$ 。图2.9显示了一个真值表，其中有多行的输出为TRUE。将每个圈出的最小项相加得到 $Y = AB + \overline{A}B$ 。

A	B	Y	minterm	minterm name
0	0	0	$\overline{A} \overline{B}$	m_0
0	1	1	$\overline{A} B$	m_1
1	0	0	$A \overline{B}$	m_2
1	1	1	$A B$	m_3

图2.9 具有多个TRUE最小项的真值表

这被称为函数的 *sum-of-products (SOP) canonical form*，因为它是乘积项（构成最小项的与运算）之和（或运算）。尽管存在多种方式表示同一函数，例如 $Y = BA + AB$ ，我们将按照最小项在真值表中出现的相同顺序对其进行排序，以便对于相同的真值表，我们始终写出相同的布尔表达式。

乘积之和规范形式也可以使用求和符号 Σ 写成 *sigma notation*。采用这种表示法，图2.9中的函数将写成：

或

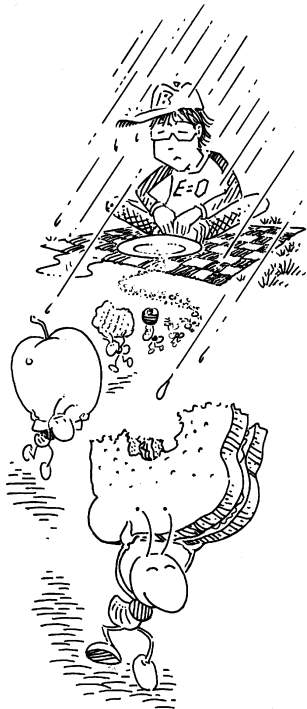
$$F(A,B) = \Sigma(m_1,m_3)$$
$$F(A,B) = \Sigma(1,3)$$

(2.2)

示例2.2 积之和（SOP）形式

Ben Bitdiddle正在野餐。如果下雨或者有蚂蚁，他就不会享受这次野餐。设计一个电路，当Ben享受野餐时输出TRUE only。

解决方案 首先，定义输入和输出。输入为A和R，分别表示是否有蚂蚁以及是否下雨。A在有蚂蚁时为真，无蚂蚁时为假。同样，R在下雨时为真，在阳光明媚时为假。输出为E，即本对野餐的享受程度。E在本享受野餐时为真，在他受苦时为假。图2.10展示了本野餐经历的真值表。



使用乘积和形式，我们将方程写为： $E = \overline{A} \overline{R}$ 或 $E = \Sigma(0)$ 。我们可以使用两个反相器和一个两输入与门来构建该方程，如图2.11(a)所示。您可能认出该真值表即为第1.5.5节中的NOR函数： $E = A \text{ NOR } R = \overline{A + R}$ 。图2.11(b)展示了NOR的实现方式。在第2.3节中，我们证明了这两个方程 $\overline{A} \overline{R}$ 和 $\overline{A + R}$ 是等价的。

乘积之和形式为任意变量数量的真值表提供了布尔方程。图2.12展示了一个随机的三输入真值表。该逻辑函数的乘积之和形式为

or

$$Y = \overline{A} \overline{B} \overline{C} + \overline{A} B \overline{C} + A \overline{B} C$$
$$Y = \Sigma(0,4,5)$$

(2.3)

A	R	E
0	0	1
0	1	0
1	0	0
1	1	0

图2.10 Ben的真值表

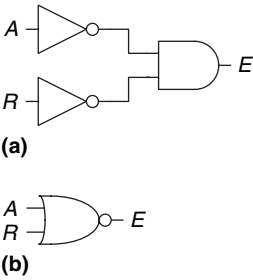


图2.11 Ben的电路

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

图2.12 随机三输入真值表

A	B	Y	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

图2.13 包含多个FALSE最大项的真值表

遗憾的是，积之和标准形式并不一定能生成最简方程。在第2.3节中，我们将展示如何使用更少的项来编写相同的函数。

2.2.3 和之积形式

表达布尔函数的另一种方式是 *product-of-sums (POS) canonical form*。真值表的每一行对应一个在该行为假的最大项。例如，两输入真值表第一行的最大项是 $(A + B)$ ，因为当 $A = 0$ 、 $B = 0$ 时， $(A + B)$ 为假。我们可以直接从真值表中写出任何电路的布尔方程，即所有输出为假的 最大项 的与运算。和之积标准形式也可以用 *pi notation* 中的乘积符号 Π 来表示。

例2.3 和之积（POS）形式

写出图2.13中真值表的积之和形式方程。

解 真值表中有两行的输出为FALSE。因此，该函数可以写成乘积之和的形式为 $Y = (A + B)(A + \overline{B})$ ，或使用 π 符号表示为 $Y = \Pi(M_0, M_2)$ 或 $Y = \Pi(,) 0 2$ 。第一个最大项 $(A + B)$ 确保当 $A = 0$ 、 $B = 0$ 时 $Y = 0$ ，因为任何值与0相与都为0。同样，第二个最大项 $(A + \overline{B})$ 确保当 $A = 1$ 、 $B = 0$ 时 $Y = 0$ 。图2.13与图2.9的真值表相同，表明同一函数可以有多种写法。

类似地，图2.10中Ben野餐的布尔方程可以通过圈出三个0行以乘积之和形式写出，得到 $E = (A + \underline{R})(\underline{A} + R)(A + R)$ 或 $(, ,) E = \Pi 1 2 3$ 。这比和之积方程 $E = AR$ 更复杂，但两个方程在逻辑上是等价的。

当输出仅在真值表的少数几行为TRUE时，积之和能生成更简短的方程；当输出仅在真值表的少数几行为FALSE时，和之积更简单。

2.3 布尔代数

在上一节中，我们学习了如何根据真值表编写布尔表达式。然而，该表达式并不一定能得到最简单的逻辑门集合。正如你使用代数简化数学方程一样，你可以使用 *Boolean algebra* 来简化布尔方程。布尔代数的规则与普通代数非常相似，但在某些情况下更为简单，因为变量只有两个可能的值：0或1。

布尔代数基于一组我们假定为正确的公理。公理在定义无法被证明的意义上是不可证明的。

表2.1 布尔代数公理

Axiom		Dual		Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binary field
A2	$\bar{0} = 1$	A2'	$\bar{1} = 0$	NOT
A3	$0 \bullet 0 = 0$	A3'	$1 + 1 = 1$	AND/OR
A4	$1 \bullet 1 = 1$	A4'	$0 + 0 = 0$	AND/OR
A5	$0 \bullet 1 = 1 \bullet 0 = 0$	A5'	$1 + 0 = 0 + 1 = 1$	AND/OR

证明。从这些公理出发，我们证明了布尔代数的所有定理。这些定理具有重要的实际意义，因为它们教会我们如何简化逻辑，从而设计出更小、成本更低的电路。

布尔代数的公理和定理遵循*dual-ity*原则。如果符号0和1以及运算符 $\bullet$ (AND)和 $+$ (OR)互换后，该语句仍然正确。我们使用撇号(')来表示语句的*dual*。

2.3.1 公理

表2.1列出了布尔代数的公理。这五条公理及其对偶定义了布尔变量以及NOT、AND和OR的含义。公理A1指出，布尔变量*B*如果不是1则为0。该公理的对偶A1'指出，该变量如果不是0则为1。A1和A1'共同告诉我们，我们是在一个由0和1构成的布尔或二元域中工作。公理A2和A2'定义了NOT运算。公理A3至A5定义了AND；它们的对偶A3'至A5'定义了OR。

2.3.2 单变量定理

表2.2中的定理T1至T5描述了如何简化涉及一个变量的方程。

*identity*定理T1指出，对于任意布尔变量*B*， B 与1 = B 。其对偶定理指出 B 或0 = B 。在硬件中，如图2.14所示，T1意味着如果两输入与门的一个输入始终为1，我们可以移除该与门，并用连接到变量输入(*B*)的导线替代。同样地，T1'意味着如果两输入或门的一个输入始终为0，我们可以用连接到*B*的导线替代该或门。通常，门电路会消耗成本、功率和延迟，因此用导线替代门电路是有益的。

null element theorem，即T2，指出*B*与0进行AND运算的结果始终为0。因此，0被称为AND运算的*null*元素，因为它会抵消任何其他输入的影响。对偶原理指出*B*与1进行OR运算的结果始终为1。因此，1是OR运算的零元素。在

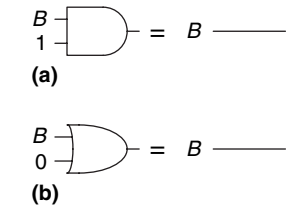


图2.14 硬件中的同一性定理：(a) T1，(b) T1'

零元素定理会导出一些看似荒诞却实际成立的结论！若落入广告商之手则尤为危险：您将获得一百万美元，否则我们邮寄一把牙刷给您。（您极大可能会收到一把牙刷。）

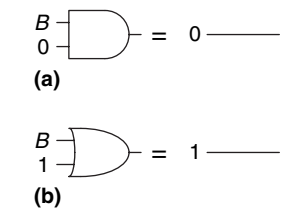


图2.15 硬件中的零元素定理：(a) T2, (b) T2'

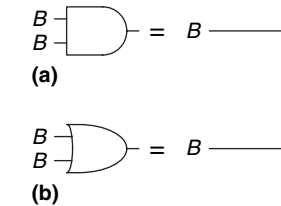


图2.16 硬件中的幂等定理：(a) T3, (b) T3'

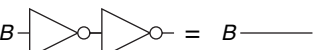


图2.17 硬件中的对合定理：T4

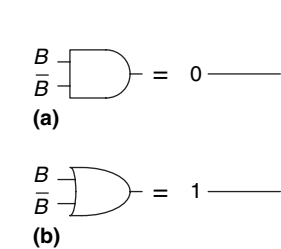


图2.18 硬件中的补码定理：(a) T5, (b) T5'

表2.2 单变量布尔定理

	Theorem		Dual	Name
T1	$B \bullet 1 = B$	T1'	$B + 0 = B$	Identity
T2	$B \bullet 0 = 0$	T2'	$B + 1 = 1$	Null Element
T3	$B \bullet B = B$	T3'	$B + B = B$	Idempotency
T4	$\overline{\overline{B}} = B$			Involution
T5	$B \bullet \bar{B} = 0$	T5'	$B + \bar{B} = 1$	Complements

硬件中，如图2.15所示，如果与门的一个输入为0，我们可以将该与门替换为一条接低电平（接0）的导线。同样地，如果或门的一个输入为1，我们可以将该或门替换为一条接高电平（接1）的导线。

*Idempotency*T3指出，变量与自身进行AND运算等于其本身。同样，变量与自身进行OR运算也等于其本身。该定理的名称源自拉丁语词根：*idem*（相同）和*potent*（幂）。这些运算返回的结果与输入相同。图2.16展示了幂等性再次允许用一根导线替代一个门电路。

*Involution*T4是一种花哨的说法，即对变量取补两次会得到原变量。在数字电子学中，两个错误反而正确。两个串联的反相器在逻辑上相互抵消，逻辑上等效于一根导线，如图2.17所示。T4的对偶形式就是其本身。

complement theorem, T5（图2.18）指出，一个变量与其补码的与运算结果为0（因为其中必有一个为0）。根据对偶性，一个变量与其补码的或运算结果为1（因为其中必有一个为1）。

2.3.3 多变量定理

表2.3中的定理T6至T12描述了如何简化涉及多个布尔变量的方程。

*Commutativity*与*associativity*、T6和T7一样，其运算方式与传统代数相同。根据交换律，AND或OR函数中输入项的*order*不会影响输出值。根据结合律，AND或OR运算中输入项的具体分组方式不会影响输出值。

distributivity theorem，即T8，与传统代数中的相同，但其对偶式T8'则不同。根据T8，AND对OR满足分配律，而根据T8'，OR对AND满足分配律。在传统代数中，乘法对加法满足分配律，但加法对乘法不满足分配律，因此 $(B + C) \times (B + D) \neq B + (C \times D)$ 。

covering、*combining*和*consensus*定理，即T9至T11，允许我们消除冗余变量。稍加思考，你应能确信这些定理是正确的。

表2.3 多变量布尔定理

Theorem		Dual		Name
T6	$B \bullet C = C \bullet B$	T6'	$B + C = C + B$	Commutativity
T7	$(B \bullet C) \bullet D = B \bullet (C \bullet D)$	T7'	$(B + C) + D = B + (C + D)$	Associativity
T8	$(B \bullet C) + (B \bullet D) = B \bullet (C + D)$	T8'	$(B + C) \bullet (B + D) = B + (C \bullet D)$	Distributivity
T9	$B \bullet (B + C) = B$	T9'	$B + (B \bullet C) = B$	Covering
T10	$(B \bullet C) + (B \bullet \overline{C}) = B$	T10'	$(B + C) \bullet (B + \overline{C}) = B$	Combining
T11	$(B \bullet C) + (\overline{B} \bullet D) + (C \bullet D)$ $= (B \bullet C) + (\overline{B} \bullet D)$	T11'	$(B + C) \bullet (\overline{B} + D) \bullet (C + D)$ $= (B + C) \bullet (\overline{B} + D)$	Consensus
T12	$\overline{B_0 \bullet B_1 \bullet B_2 \dots}$ $= (\overline{B_0} + \overline{B_1} + \overline{B_2} \dots)$	T12'	$\overline{B_0 + B_1 + B_2 \dots}$ $= (\overline{B_0} \bullet \overline{B_1} \bullet \overline{B_2} \dots)$	De Morgan's Theorem

De Morgan's Theorem, T12, 是数字设计中一个特别强大的工具。该定理说明, 所有项乘积的补等于各项补的和。同样地, 所有项和的补等于各项补的乘积。

根据德摩根定理, 与非门等效于输入取反的或门。同样, 或非门等效于输入取反的与门。图2.19展示了与非门和或非门的*De Morgan* 和 *equivalent gates*。每个功能对应的两个符号称为*duals*。它们在逻辑上等效, 可以互换使用。

反相圈被称为*bubble*。直观上, 你可以想象“推动”一个气泡穿过门会导致它从另一端出来。

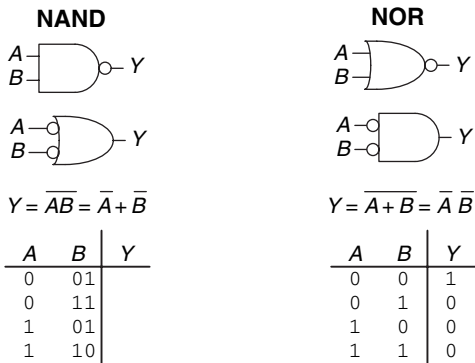


图2.19 德摩根等效门



奥古斯都·德·摩根, 1871年逝世, 英国数学家, 出生于印度。一只眼失明。10岁时父亲去世。16岁进入剑桥大学三一学院, 22岁被任命为新成立的伦敦大学数学教授。在数学诸多领域著述颇丰, 包括逻辑学、代数学和悖论。月球上的德·摩根环形山以他的名字命名。他曾为自己的出生年份提出一个谜题: “我在 x^2 年时是 x 岁。”

注意， $\overline{AB}$ 等于 $\overline{A}B$ ， $\overline{AB}$ 不等于 $A\overline{B}$ 。

侧边翻转门的主体，将AND变为OR，反之亦然。例如，图2.19中的与非门由一个输出端带气泡的AND主体组成。将气泡向左推，则得到一个输入端带气泡的OR主体。气泡推动的基本规则如下：

将气泡向后推（从输出端）或向前推（从输入端）会将门的主体从AND变为OR，反之亦然。► 将气泡从输出端推回至输入端会在所有门输入端上放置气泡。► 将all门输入端上的气泡向前推向输出端会在输出端上放置一个气泡。

第2.5.2节使用气泡推演来帮助分析电路。

示例2.4 推导和之积形式

图2.20展示了布尔函数Y及其补函数Y的真值表。利用德摩根定理，从Y的与或式推导出Y的或与标准形式。

A	B	Y	$\overline{Y}$
0	0	0	1
0	1	0	1
1	0	1	0
1	1	1	0

图2.20 显示Y和Y的真值表

解 图2.21展示了Y中包含的最小项（圈出）。Y的积之和标准形式为

$$\overline{Y} = \overline{A}B + A\overline{B}$$

(2.4)

对两边取补集并两次应用德摩根定理，我们得到

A	B	Y	$\overline{Y}$	minterm
0	0	0	1	$\overline{A}\overline{B}$
0	1	0	1	$\overline{A}B$
1	0	1	0	$A\overline{B}$
1	1	1	0	AB

图2.21 展示Y的最小项的真值表

$$\overline{\overline{Y}} = Y = \overline{\overline{A}B + A\overline{B}} = (\overline{\overline{A}B})(\overline{A\overline{B}}) = (A + B)(A + \overline{B})$$

(2.5)

2.3.4 这一切背后的真相

好奇的读者可能会想知道如何证明一个定理是正确的。在布尔代数中，证明含有有限个变量的定理很简单：只需证明该定理对这些变量的所有可能取值都成立。这种方法称为*perfect induction*，可以通过真值表来实现。

示例2.5 使用完全归纳法证明一致性定理

证明表2.3中的一致性定理T11。

解：检查方程两边关于B、C和D的所有八种组合。图2.22中的真值表展示了这些组合。由于所有情况下 $BC + BD + CD = BC + BD$ 均成立，该定理得证。

B	C	D	$BC + \overline{B}D + CD$	$BC + \overline{B}D$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1

图2.22 真值表
证明 T11

2.3.5 简化方程

布尔代数的定理帮助我们简化布尔方程。例如，考虑图2.9真值表中的乘积之和表达式： $Y = AB + \overline{A}B$ 。根据合并定理（T10），该方程简化为 $Y = B$ 。这在查看真值表时可能显而易见。通常，简化更复杂的方程可能需要多个步骤。

简化乘积和方程的基本原理是利用关系式 $PA + PA = P$ 合并项，其中 P 可以是任意项。一个方程能简化到什么程度？我们将乘积和形式的方程定义为 $minimized$ ，如果它使用了最少的蕴含项。当存在多个具有相同数量蕴含项的方程时，最简方程是文字数量最少的那一个。

蕴含项若不能与方程中的任何其他蕴含项合并以形成具有更少文字的新蕴含项，则称为 $prime\ implicant$ 。最小方程中的蕴含项必须全部是质蕴含项，否则它们可以被合并以减少文字数量。

表2.2和2.3列出了布尔公理和定理的基本形式，但它们也适用于更复杂的项。例如，我们将组合定理应用于表达式 $(A + BC)D + A + BC\overline{D}$ ，以得到

—

—

—

()

—

A + BC

。

示例2.6 方程最小化

最小化方程2.3: $ABC + \overline{A}BC + A\overline{B}C$ 。

解 我们从原始方程开始，逐步应用布尔定理，如表2.4所示。

至此，我们是否已完全简化了方程？让我们仔细审视。从原始方程来看，最小项 ABC 和 $\overline{A}BC$ 仅在变量 A 上不同，因此我们将它们合并为 BC 。然而，若观察原始方程，我们会发现最后两个最小项 ABC 和 $A\overline{B}C$ 也仅相差一个文字（ C 和 $\overline{C}$ ）。因此，采用相同方法，我们本可将这两个最小项合并为最小项 AB 。我们称蕴含项 BC 和 AB share为最小项 ABC 。

那么，我们是否只能简化其中一个最小项对，还是可以同时简化两者？利用幂等定理，我们可以根据需要进行多次复制项： $B = B + B + B + B \cdots$ 。运用这一原理，我们将方程完全简化为两个质蕴含项，即 $BC + AB$ ，如表2.5所示。

表2.4 方程最小化

Step	Equation	Justification
	$\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$	
1	$\overline{B}\overline{C}(\overline{A} + A) + A\overline{B}C$	T8: Distributivity
2	$\overline{B}\overline{C}(1) + A\overline{B}C$	T5: Complements
3	$\overline{B}\overline{C} + A\overline{B}C$	T1: Identity

表2.5 改进的方程最小化

Step	Equation	Justification
	$\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$	
1	$\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$	T3: Idempotency
2	$\overline{B}\overline{C}(\overline{A} + A) + A\overline{B}(\overline{C} + C)$	T8: Distributivity
3	$\overline{B}\overline{C}(1) + A\overline{B}(1)$	T5: Complements
4	$\overline{B}\overline{C} + A\overline{B}$	T1: Identity

虽然这有点反直觉，但*expanding*一个蕴含项（例如将 AB 转换为 $ABC + ABC$ ）有时在简化方程中很有用。通过这样做，你可以重复使用其中一个展开的最小项，以便与另一个最小项进行合并（共享）。

你可能已经注意到，使用布尔代数定理完全简化一个布尔方程可能需要一些试错。第2.7节描述了一种称为*Karnaugh maps*的系统化技术，可以使这一过程更加容易。

既然布尔方程在逻辑上等价，为何还要费心化简？化简能减少用于物理实现该函数的门电路数量，从而使其更小、更便宜，甚至可能更快。下一节将介绍如何用逻辑门实现布尔方程。

2.4 从逻辑到门电路

schematic 是数字电路的示意图，展示了各元件及其连接导线。例如，图2.23中的原理图展示了我们最常用的逻辑函数（即公式2.3）的一种可能硬件实现方式：

$$Y = \overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$$

(2.3)

本教材附带的实验（见前言）展示了如何使用 *computer-aided*（CAD）工具来设计、仿真和测试数字电路。

esign

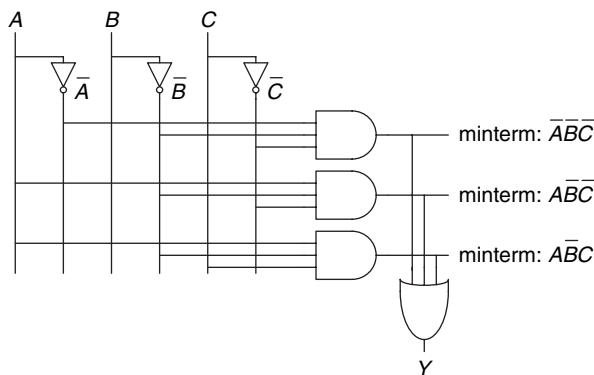


图2.23 $Y = ABC + ABC + ABC$ 示意图

通过以一致的方式绘制原理图，我们使其更易于阅读和调试。我们通常遵循以下准则：

- ▶ 输入位于原理图的左侧（或顶部）。▶ 输出位于原理图的右侧（或底部）。
- ▶ 尽可能让门电路从左向右流动。
- ▶ 使用直线导线优于使用带有多个拐角的导线（锯齿状导线会浪费脑力去追踪导线，而非思考电路的功能）。
- ▶ 导线始终在T形连接点处连接。
- ▶ 导线交叉处的圆点表示导线之间的连接。
- ▶ 导线交叉处若无圆点则表示无连接。

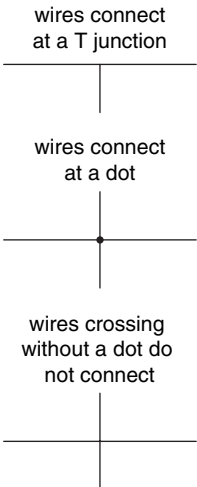


图2.24 导线连接

最后三条准则如图2.24所示。

任何以乘积和形式表示的布尔方程都可以按照类似于图2.23的系统化方式绘制成原理图。首先，为输入绘制列。如有必要，在相邻列中放置反相器以提供互补输入。为每个最小项绘制与门行。然后，对于每个输出，绘制一个或门，连接到与该输出相关的最小项。这种风格称为 *programmable logic array (PLA)*，因为反相器、与门和或门以系统化的方式排列。PLA将在第5.6节进一步讨论。

图2.25展示了我们在例2.6中通过布尔代数得到的简化方程的实现。注意，简化后的电路硬件比图2.23中的电路少得多，而且由于使用了输入更少的门电路，它可能也更快。

我们可以利用反相门进一步减少门数量（尽管只减少一个反相器）。注意到BC是一个输入反相的与门。图2.26展示了利用这一优化消除C上反相器的原理图。回顾德摩根定律，

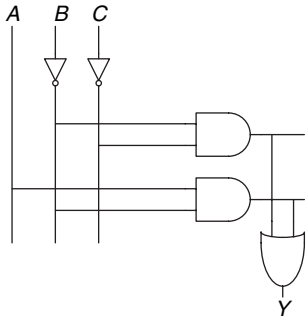


图2.25 $Y = BC + AB$ 示意图

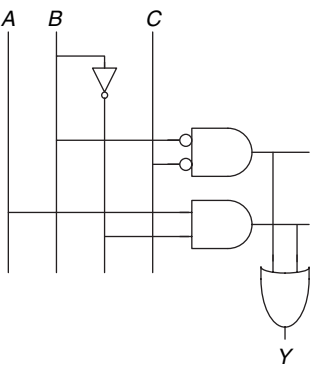


图2.26 使用更少门电路的示意图

定理，具有反相输入的与门等效于或非门。根据实现技术，使用最少的门电路或优先使用特定类型的门电路可能更经济。例如，在CMOS实现中，与非门和或非门比与门和或门更受青睐。

许多电路具有多个输出，每个输出计算输入的一个独立布尔函数。我们可以为每个输出编写单独的真值表，但通常更方便的做法是将所有输出写在同一张真值表上，并绘制包含所有输出的单一原理图。

示例2.7 多输出电路

院长、系主任、助教和宿舍社交主席偶尔会使用礼堂。不幸的是，他们偶尔会发生冲突，导致诸如院长与固执的董事们举行的筹款会议与宿舍的BTB¹派对同时进行之类的灾难。Alyssa P. Hacker被请来设计一个房间预订系统。

该系统有四个输入 A_3 、 $\dots$ 、 A_0 和四个输出 Y_3 、 $\dots$ 、 Y_0 。这些信号也可表示为 $A_{3:0}$ 和 $Y_{3:0}$ 。每个用户在申请次日使用礼堂时发出其输入信号。系统最多只发出一个输出信号，将礼堂授予优先级最高的用户。出资建设该系统的院长要求最高优先级（3）。系主任、助教和宿舍社交主席的优先级依次递减。

为该系统编写真值表和布尔方程。绘制一个实现此功能的电路图。

解 该函数称为四输入*priority circuit*。其符号和真值表如图2.27所示。

我们可以将每个输出写成乘积和的形式，并用布尔代数简化方程。然而，根据功能描述（以及真值表），简化后的方程一目了然：当 A_3 有效时， Y_3 为真，因此 $Y_3 = A_3$ 。如果 A_2 有效且 A_3 无效，则 Y_2 为真，因此 $Y_2 = \overline{A_3} A_2$ 。当 A_1 有效且优先级更高的输入均无效时， $Y_1 = \overline{A_3} \overline{A_2} A_1$ 为真。而只要 A_0 有效且没有其他输入有效， Y_0 即为真： $Y_0 = \overline{A_3} \overline{A_2} \overline{A_1} A_0$ 。原理图如图2.28所示。经验丰富的设计者通常能通过观察直接实现逻辑电路。给定清晰的规格说明，只需将文字转化为方程，再将方程转化为门电路即可。

注意，如果在优先级电路中断言了 A_3 ，则输出*don't care*与其他输入无关。我们使用符号x来描述输入。

¹Black light, twinkies, and beer.

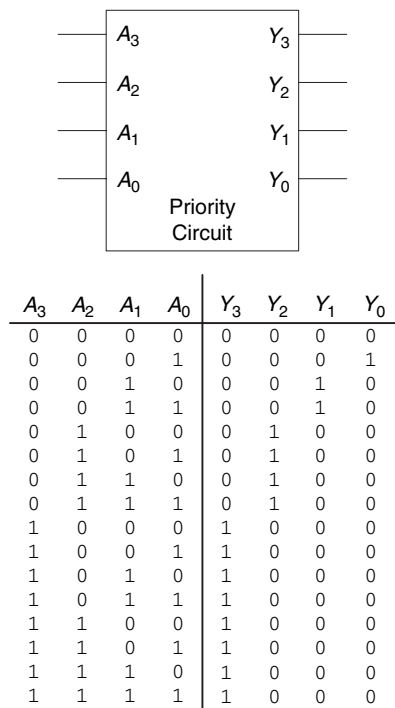


图2.27 优先级电路

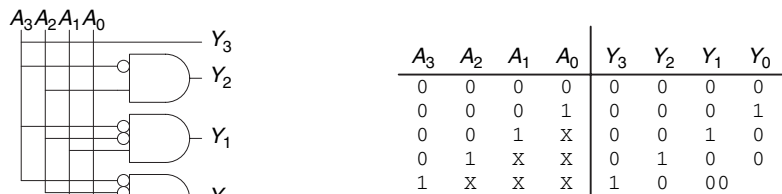


图2.29 带无关项 (X) 的优先电路真值表

图2.28 优先电路原理图

输出不关心的内容。图2.29展示了四输入优先电路的真值表在引入无关项后变得小得多。根据这个真值表，我们可以通过忽略带有X的输入，轻松地读出乘积和形式的布尔方程。无关项也可能出现在真值表的输出中，我们将在2.7.3节中看到这一点。

2.5 多级组合逻辑

采用“积之和”形式的逻辑称为two-level logic，因为它由连接到一级与门再连接到一级或门的文字组成。设计人员通常构建超过两级的电路。

X是一个重载符号，在真值表中表示“无关项”，在逻辑仿真中表示“竞争”（见第2.6.1节）。请结合上下文理解，以免混淆含义。部分作者为避免歧义，使用D或?来表示“无关项”。

逻辑门。这些多级组合电路可能比其两级对应电路使用更少的硬件。气泡推演在分析和设计多级电路时尤其有用。

2.5.1 硬件缩减

某些逻辑函数在使用两级逻辑构建时需要大量的硬件。一个显著的例子是多变量的异或函数。例如，考虑使用我们目前研究的两级技术构建一个三输入异或函数。

回想一下，当奇数个输入为真时，N输入异或门输出为真。图2.30展示了三输入异或门的真值表，其中产生真输出的行已用圆圈标出。根据真值表，我们得出式2.6中积之和形式的布尔方程。遗憾的是，无法将该方程简化为更少的蕴含项。

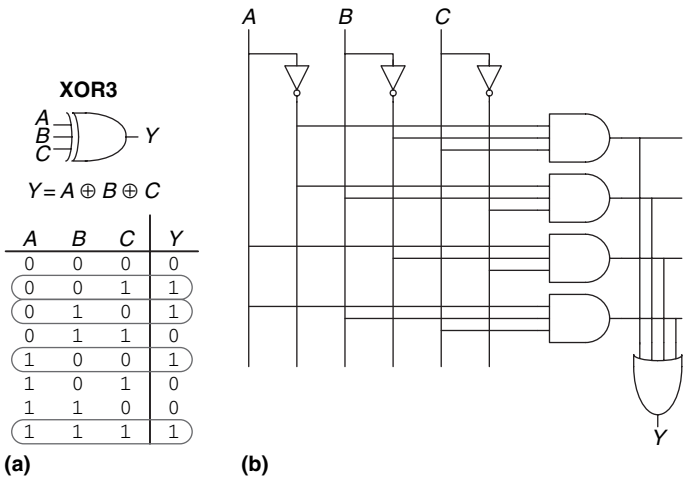
$$Y = \overline{A}\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC$$

(2.6)

另一方面， $A \oplus B \oplus C = (A \oplus B) \oplus C$ （如有疑问，可通过完全归纳法自行证明）。因此，三输入异或门可由两级两输入异或门级联构成，如图2.31所示。

类似地，一个八输入异或门若采用两级与或式实现，需要128个八输入与门和一个128输入或门。更好的选择是使用两输入异或门构成的树形结构，如图2.32所示。

图2.30 三输入异或：(a)功能规格和(b)两级逻辑实现



为特定逻辑函数选择最佳的多级实现并非一个简单的过程。此外，“最佳”有多种含义：门数最少、速度最快、设计时间最短、成本最低、功耗最小。在第5章中，您将看到，在一种技术中“最佳”的电路在另一种技术中未必是最佳的。例如，我们一直使用与门和或门，但在CMOS中，与非门和或非门效率更高。通过一些经验，您会发现对于大多数电路，您可以通过观察来创建良好的多级设计。在本书后续章节学习电路示例时，您将积累一些这样的经验。在学习过程中，请探索各种设计选项并思考其中的权衡。计算机辅助设计（CAD）工具也可用于搜索庞大的多级设计空间，并寻找在给定可用构建模块下最符合您约束条件的设计。

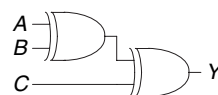


图2.31 使用两个二输入异或门构成的三输入异或门

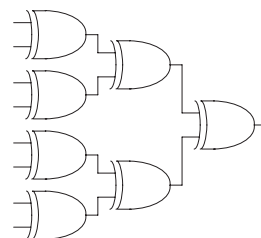


图2.32 使用七个两输入异或门实现的八输入异或门

2.5.2 气泡推挤

您可能还记得第1.7.6节中提到，CMOS电路更倾向于使用与非门和或非门，而非与门和或门。但通过观察从包含与非门和或非门的多级电路中直接解读逻辑表达式可能会相当复杂。图2.33展示了一个多级电路，其功能无法通过直接观察立即明确。气泡推演是一种有用的方法，可以重新绘制这些电路，使气泡相互抵消，从而更容易确定其功能。基于第2.3.3节中的原理，气泡推演的指导原则如下：

- 从电路的输出端开始，向输入端方向进行。
- 将最终输出上的任何气泡推回输入端，以便你能读取一个以输出（例如Y）而非输出补集（ $\bar{Y}$ ）表示的方程。
- 从后向前推导，以气泡可相互抵消的形式绘制每个逻辑门。若当前门电路有输入气泡，则前一级门电路应绘制为带输出气泡；若当前门电路无输入气泡，则前一级门电路应绘制为不带输出气泡。

图2.34展示了如何根据气泡推挤规则重新绘制图2.33。从输出端Y开始，NAND门的输出端有一个我们想要消除的气泡。我们将输出气泡向后推，形成具有反相输入的OR门，如

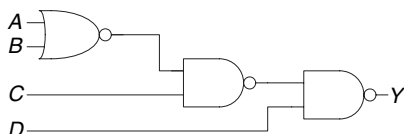


图2.33 使用与非门和或非门的多级电路

图2.34 气泡推动电路

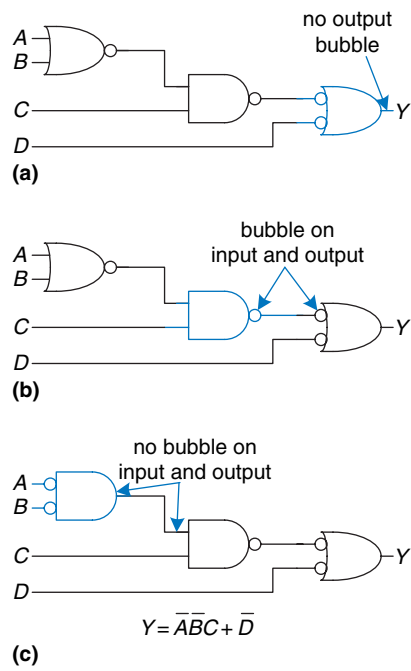
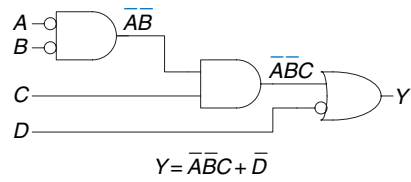


图2.34(a)。向左操作，最右侧的门有一个输入气泡，与中间与非门的输出气泡相抵消，因此无需更改，如图2.34(b)所示。中间的门没有输入气泡，因此我们将最左侧的门转换为无输出气泡，如图2.34(c)所示。现在，电路中除输入外所有气泡均被抵消，因此可以通过检查以与门和或门的形式读取真值或互补输入的函数： $Y = \overline{A}BC + \overline{D}$ 。

为强调最后一点，图2.35展示了一个逻辑上与图2.34等效的电路。内部节点的功能以蓝色标注。由于串联的圆圈相互抵消，我们可以忽略中间门输出端和最右侧门一个输入端的圆圈，从而得到图2.35中逻辑等效的电路。

图2.35 逻辑等效的推泡电路



示例2.8 CMOS逻辑中的气泡推挤

大多数设计师习惯于使用与门和或门进行思考，但假设你希望用CMOS逻辑实现图2.36中的电路，而CMOS逻辑更倾向于使用与非门和或非门。请使用气泡推演法将电路转换为与非门、或非门和反相器。

解决方案一种暴力解法是将每个与门替换为一个与非门和一个反相器，每个或门替换为一个或非门和一个反相器，如图2.37所示。这需要八个门。注意，反相器绘制时气泡在前端而非后端，以强调气泡如何与前面的反相门相互抵消。

为了获得更好的解决方案，可以观察到，在门的输出端和下一级门的输入端添加气泡不会改变功能，如图2.38(a)所示。最终的与门被转换为一个与非门和一个反相器，如图2.38(b)所示。该方案仅需五个门。

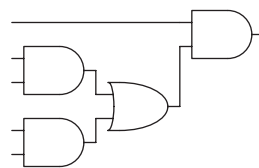


图2.36 使用与门和或门的电路

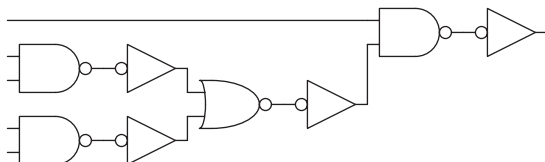


图2.37 使用与非门和或非门的糟糕电路

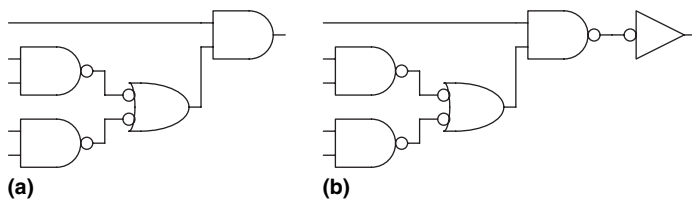


图2.38 使用与非门和或非门的改进电路

2.6 X与Z，天哪

布尔代数仅限于0和1。然而，实际电路还可能存在非法值和浮空值，分别用符号X和Z表示。

2.6.1 非法值：X

符号X表示电路节点具有*unknown*或*illegal*值。这种情况通常发生在该节点同时被驱动为0和1时。图2.39展示了一个节点Y同时被驱动为高电平和低电平的案例。这种被称为*contention*的情况被视为一种错误。

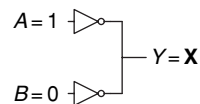


图2.39 存在竞争现象的电路

必须避免。存在争用的节点上的实际电压可能介于0和 V_{DD} 之间，具体取决于驱动高电平和低电平的门电路的相对强度。该电压通常（但并非总是）处于禁止区域。争用还可能导致大量电流在相互对抗的门电路之间流动，从而使电路发热并可能损坏。

X 值有时也被电路模拟器用来表示未初始化的值。例如，如果你忘记指定某个输入的值，模拟器可能会假定其为 X，以提醒你注意该问题。

如第2.4节所述，数字设计者在真值表中也使用符号X表示“无关”值。请务必不要混淆这两种含义。当X出现在真值表中时，表示该变量在真值表中的值不重要（可以是0或1）。当X出现在电路中时，表示该电路节点具有未知或非法值。

2.6.2 浮动值：Z

符号Z表示节点既未被驱动为高电平，也未被驱动为低电平。该节点被称为floating、high impedance或high Z。一个典型的误解是，浮空或未被驱动节点等同于逻辑0。实际上，浮空节点可能为0，可能为1，也可能处于两者之间的某个电压值，具体取决于系统的历史状态。只要当节点值对电路运行相关时，有其他电路元件将其驱动至有效逻辑电平，浮空节点并不总意味着电路存在错误。

产生浮动节点的一种常见方式是忘记将电压连接到电路输入，或假设未连接的输入与值为0的输入相同。这种错误可能导致电路行为异常，因为浮动输入会随机从0变为1。事实上，触摸电路可能就足以通过身体静电触发这种变化。我们曾见过一些电路，只有在学生用手指按住芯片时才能正常运行。

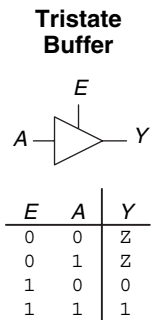


图2.40 三态缓冲器

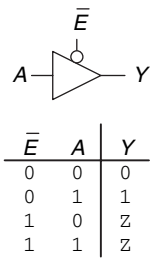


图2.41 具有低电平有效使能的三态缓冲器

如图2.40所示，tristate buffer具有三种可能的输出状态：高电平（1）、低电平（0）和浮空（Z）。三态缓冲器具有输入A、输出Y和enable E。当使能信号为真时，三态缓冲器作为简单缓冲器工作，将输入值传输至输出。当使能信号为假时，输出被允许处于浮空状态（Z）。

图2.40中的三态缓冲器具有active high使能。即，当使能为高电平（1）时，缓冲器被使能。图2.41展示了一个

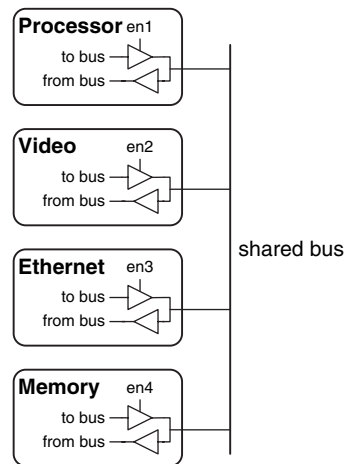


图2.42 连接多个芯片的三态总线

具有*active low*使能的三态缓冲器。当使能为低电平（0）时，缓冲器被启用。我们通过在输入线上加一个气泡来表示该信号为低电平有效。通常通过在信号名称上方画一条横线（*E*），或在名称后附加字母“b”或“bar”（*Eb*或 $\bar{E}$ ）来表示低电平有效输入。

三态缓冲器常用于连接多个芯片的**busses**上。例如，微处理器、视频控制器和以太网控制器可能都需要与个人计算机中的内存系统通信。每个芯片可以使用三态缓冲器连接到共享内存总线，如图2.42所示。每次只允许一个芯片断言其使能信号以驱动值到总线上。其他芯片必须产生浮空输出，以免与正在与内存通信的芯片发生冲突。任何芯片都可以随时从共享总线读取信息。这种三态总线曾经很常见。然而，在现代计算机中，通过*point-to-point links*可以实现更高的速度，其中芯片之间直接连接，而不是通过共享总线。

2.7 卡诺图

在使用布尔代数对布尔方程进行多次化简后，你会意识到，如果不小心，有时最终得到的会是一个完全不同的*different*方程，而非简化后的方程。*Karnaugh maps* (*K-maps*) 是一种用于简化布尔方程的图形化方法。它们由贝尔实验室的电信工程师莫里斯·卡诺于1953年发明。卡诺图的工作原理

莫里斯·卡诺，1924–1948年毕业于纽约市立学院，获物理学学士学位，1952年在耶鲁大学获得物理学博士学位。

1952年至1993年在贝尔实验室和IBM工作，1980年至1999年在纽约理工学院担任计算机科学教授。

格雷码由贝尔实验室研究员弗兰克·格雷于1953年获得专利（美国专利2,632,058）。这种编码在机械编码器中尤为实用，因为轻微的对准偏差仅会导致单个比特的错误。

格雷码可推广至任意位数。例如，一个3位格雷码序列为：

000, 001, 011, 010, 110, 111, 101, 100

刘易斯·卡罗尔在1879年的 *Vanity Fair* 中提出了一个相关的谜题。

“谜题的规则相当简单。给出两个长度相同的单词；谜题的目标是通过插入其他单词将它们连接起来，每个单词与下一个单词仅有一个字母不同。也就是说，可以改变其中一个给定单词的一个字母，然后改变由此得到的单词中的一个字母，依此类推，直到我们到达另一个给定的单词。”

例如，从船舶到码头：

SHIP, SLIP, SLOP, SLO
T, SOOT, LOOT, LOOK
, LOCK, DOCK.

你能找到更短的序列吗？

对于最多包含四个变量的问题，它们更为重要的是提供了操作布尔方程的洞察。

回想一下，逻辑最小化涉及合并项。包含蕴涵项 P 以及某个变量 A 的真形式和互补形式的两个项被合并以消除 A $\overline{PA}$ $\overline{PA}$ P ：。
 $+ =$ 卡诺图通过将这些可合并的项在网格中相邻放置，使其易于识别。
。

图2.43展示了一个三输入函数的真值表和K-map。K-map的顶行给出了 A 和 B 输入的四种可能取值。左列给出了 C 输入的两种可能取值。K-map中的每个方格对应真值表中的一行，并包含该行输出 Y 的值。例如，左上角的方格对应真值表的第一行，表示当 $ABC = 000$ 时输出值 $Y = 1$ 。正如真值表中的每一行，K-map中的每个方格代表一个最小项。为了解释，图2.43(c)显示了K-map中每个方格对应的最小项。

每个方格（即最小项）与相邻方格仅相差一个变量的变化。这意味着相邻方格共享除一个变量外的所有相同文字，该变量在一个方格中以原变量形式出现，在另一个方格中以互补形式出现。例如，表示最小项 ABC 和 $\overline{A}BC$ 的方格相邻，且仅变量 A 不同。你可能注意到，顶行中的 A 和 B 组合按特殊顺序排列：00、01、11、10。这种顺序称为 *Gray code*。它与普通二进制顺序（00、01、10、11）的不同之处在于，相邻条目仅在一个变量上不同。例如，01:11仅将 A 从0变为1，而01:10会将 A 从0变为1，并将 B 从1变为0。因此，按二进制顺序编写组合无法实现我们期望的相邻方格仅在一个变量上不同的特性。

K-map也会“环绕”。最右边的方格实际上与最左边的方格相邻，因为它们仅在一个变量 A 上不同。换句话说，你可以将这张图卷成一个圆柱体，然后将圆柱体的两端连接起来形成一个环面（即甜甜圈形状），这样仍然能保证相邻的方格仅在一个变量上不同。

2.7.1 循环思维

在图2.43的K-map中，方程中仅存在两个最小项 ABC 和 $\overline{A}BC$ ，如左列中的1所示。从K-map读取最小项完全等同于直接从真值表中读取乘积和形式的方程。

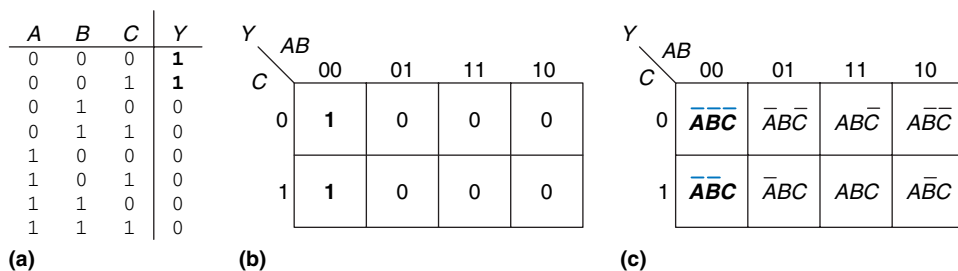


图2.43 三输入函数: (a) 真值表, (b) 卡诺图, (c) 显示最小项的卡诺图

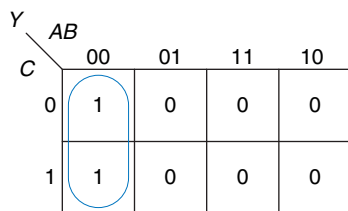


Figure 2.44 K-map minimization

如前所述, 我们可以使用布尔代数来最小化积之和形式的方程。

$$Y = \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} = \overline{A}\overline{B}(\overline{C} + C) = \overline{A}\overline{B} \quad (2.7)$$

K-map 通过将相邻方格中的 *circling* 1 进行圈选, 帮助我们以图形方式完成这种简化, 如图 2.44 所示。对于每个圈, 我们写出对应的蕴涵项。回顾第 2.2 节, 蕴涵项是一个或多个文字 (literals) 的乘积。在圈中同时出现真值 *and* 和互补形式的变量会被排除在蕴涵项之外。在这种情况下, 变量 C 在圈中同时具有其真值形式 (1) 和互补形式 (0), 因此我们不将其包含在蕴涵项中。换句话说, 当 $A = B = 0$ 时, Y 为真, 与 C 无关。因此, 蕴涵项为 AB 。K-map 给出了我们使用布尔代数得到的相同结果。

2.7.2 使用卡诺图进行逻辑化简

K-map 提供了一种直观简便的方法来最小化逻辑。只需在地图中圈出所有由 1 组成的矩形块, 且使用的圈数尽可能少。每个圈应尽可能大。然后, 读出被圈出的蕴涵项。

更正式地说, 回忆一下, 布尔方程在写成质蕴涵项数量最少的和形式时被最小化。卡诺图上的每个圈代表一个蕴涵项。尽可能大的圈就是质蕴涵项。

例如，在图2.44的卡诺图中， ABC 和 $\overline{A}\overline{B}C$ 是蕴含项，而 $\overline{A}B\overline{C}$ 是质蕴含项。在该卡诺图中，只有 AB 是质蕴含项。从卡诺图中寻找最简方程的原则如下：

- 使用尽可能少的圆圈覆盖所有1。
- ▶ 每个圆圈内的所有方格必须包含1。
- ▶ 每个圆圈必须覆盖一个矩形区域，该区域在每个方向上的方格数为2的幂（即1、2或4）。
- ▶ 每个圆圈应尽可能大。
- ▶ 圆圈可以环绕K-map的边缘。
- ▶ K-map中的1可以被多次圈选，如果这样做能使用更少的圆圈。

例2.9 三变量函数的最小化
使用K-MAP

假设我们有函数 $Y = F(A、B、C)$ ，其卡诺图如图2.45所示。使用卡诺图化简该方程。

解决方案 在K-map中圈出1，尽可能使用最少的圈，如图2.46所示。K-map中的每个圈代表一个质蕴含项，每个圈的维度是2的幂次（ $2^1 \times 1$ 和 $2^1 \times 2$ ）。我们通过写出那些在圈中仅以原变量或仅以补变量形式出现的变量，来为每个圈形成质蕴含项。

例如，在 2×1 的圈中， B 的真值形式和互补形式都包含在该圈内，因此我们在质蕴含项中 $do not$ 包含 B 。然而，该圈中仅包含 A 的真值形式（ $\overline{A}$ ）和 C 的互补形式，因此我们在质蕴含项 $\overline{A}C$ 中包含这些变量。类似地， 2×2 的圈覆盖了所有满足 $B = 0$ 的方格，因此质蕴含项为 $\overline{B}$ 。

注意右上角方块（最小项）被覆盖两次，以使质蕴含圈尽可能大。正如我们在布尔代数技巧中所见，这相当于共享一个最小项以减小蕴含项的大小。同时注意，覆盖四个方块的圈环绕卡诺图的边缘。

Y C	AB			
	00	01	11	10
0	1	0	1	1
1	1	0	0	1

图2.45 {v*}的卡诺图
示例2.9

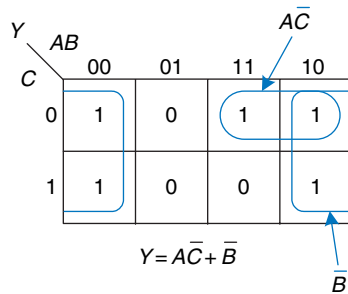


图2.46 示例2.9的解

示例2.10 七段数码管显示译码器

一个 *seven-segment display decoder* 接收4位数据输入 $D_{3:0}$ ，并产生七个输出来控制发光二极管，以显示从0到9的数字。这七个输出通常称为段 a 至 g ，或 S_a – S_g ，如图2.47所定义。数字如图2.48所示。为这些输出编写真值表，并使用卡诺图找出输出 S_a 和 S_b 的布尔方程。假设非法输入值（10–15）产生空白显示。

解决方案 真值表如表2.6所示。例如，输入0000应点亮除 S_g 外的所有段。

七个输出中的每一个都是四个变量的独立函数。输出 S_a 和 S_b 的卡诺图如图2.49所示。请记住，相邻方格只能在一个变量上不同，因此我们按照格雷码顺序标记行和列：00、01、11、10。在将输出值填入方格时，也要注意记住这一顺序。

接下来，圈出质蕴含项。使用最少数量的圈来覆盖所有的1。圈可以环绕边缘（垂直 *and* 水平），且一个1可以被多次圈出。图2.50展示了质蕴含项及简化后的布尔方程。

注意，质蕴含项的最小集合并不唯一。例如，在 S_a 卡诺图中，圈选了0000项和1000项以生成 $D_2 D_1 D_0$ 最小项。也可以改为圈选0010项，从而生成 $D_3 D_2 D_0$ 最小项，如图2.51中虚线所示。

图2.52（见第80页）展示了一个常见错误：选择了一个非质蕴含项来覆盖左上角的1。这个最小项 $D_3 D_2 D_1 D_0$ 产生了一个非最小化的乘积之和方程 *not*。该最小项本可以与相邻的任一最小项合并，形成一个更大的圈，正如前两幅图所示。

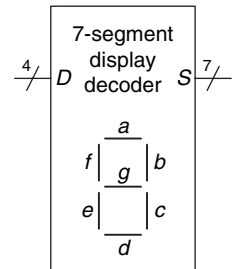


图2.47 七段数码管显示解码器图标

图2.48 七段数码管显示数字

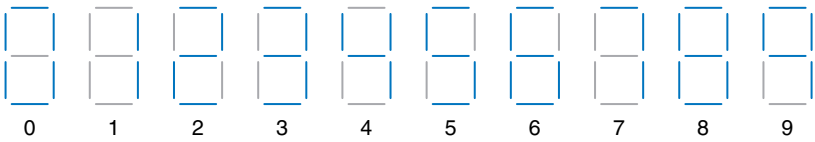
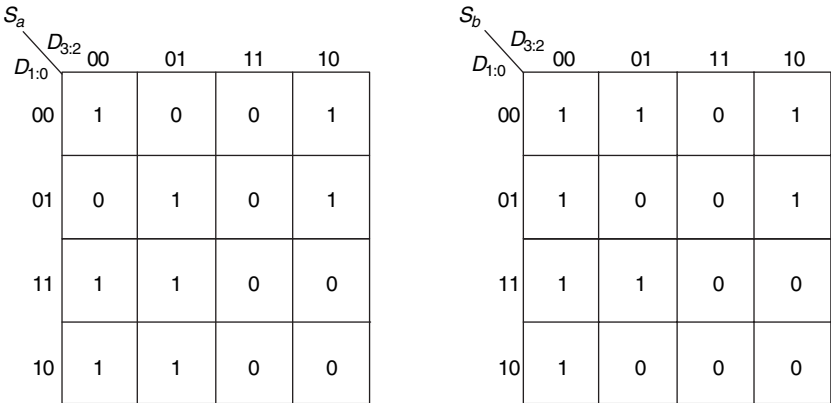


表2.6 七段数码管译码器真值表

$D_{3:0}$	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	0	0	1	1
others	0	0	0	0	0	0	0

图2.49 S_a 和 S_b 的卡诺图



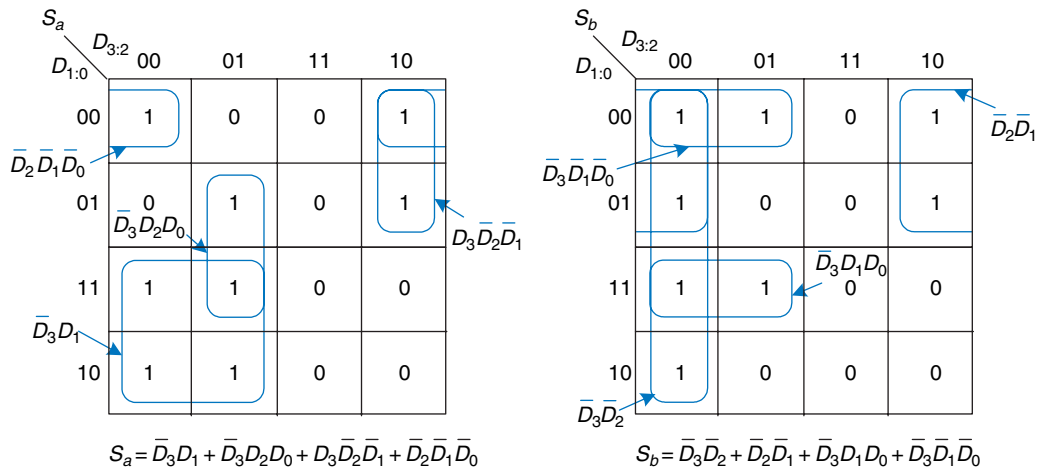
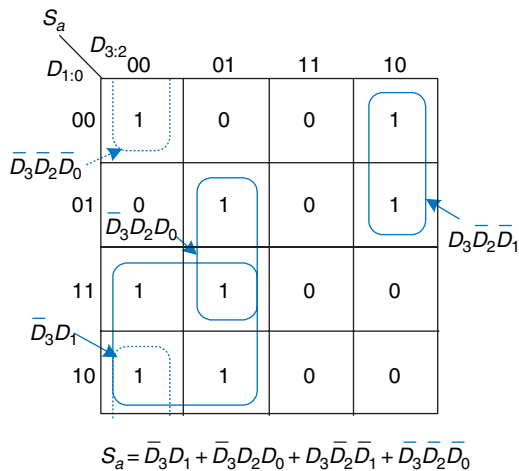


图2.50 示例2.10的K图解法

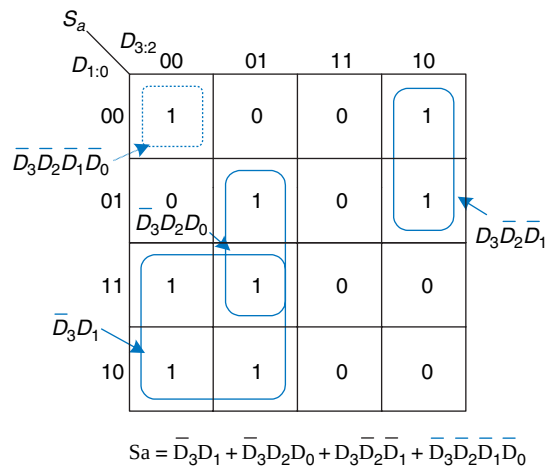
图2.51 S_a 的另一种卡诺图，展示了不同的质蕴含项集合

2.7.3 无关项

回忆一下，在2.4节中引入了真值表输入的“无关项”条目，以减少当某些变量不影响输出时表格的行数。它们用符号X表示，意味着该条目可以是0或1。

无关项也会出现在真值表输出中，此时输出值不重要，或者对应的输入组合永远不会出现。

图2.52 S_a 的另一种卡诺图，显示了不正确的非质蕴含项



发生。此类输出可由设计者自行决定视为0或1。

在卡诺图中，无关项（X）允许进一步简化逻辑。如果它们有助于用更少或更大的圈覆盖1，则可以将其圈出，但如果无帮助，则不必圈出。

示例2.11 带无关项的七段显示译码器

重复示例2.10，如果我们不关心非法输入值10到15的输出值。

解 K图如图2.53所示，其中X项表示无关项。由于无关项可为0或1，若圈入无关项能使1的覆盖圈更少或更大，则将其圈入。被圈入的无关项视为1，未被圈入的无关项视为0。注意，对于段 S_a ，一个环绕四角的 2×2 方格被圈出。利用无关项可大幅简化逻辑。

2.7.4 整体概览

布尔代数和卡诺图是两种逻辑简化方法。最终目标是找到一种低成本的方式来实现特定的逻辑功能。

在现代工程实践中，称为*logic synthesizers*的计算机程序能够根据逻辑功能的描述生成简化电路，我们将在第4章中看到这一点。对于大规模问题，逻辑综合器比人类高效得多。而对于小规模问题，稍有经验的人可以通过观察找到良好的解决方案。这两种方法都

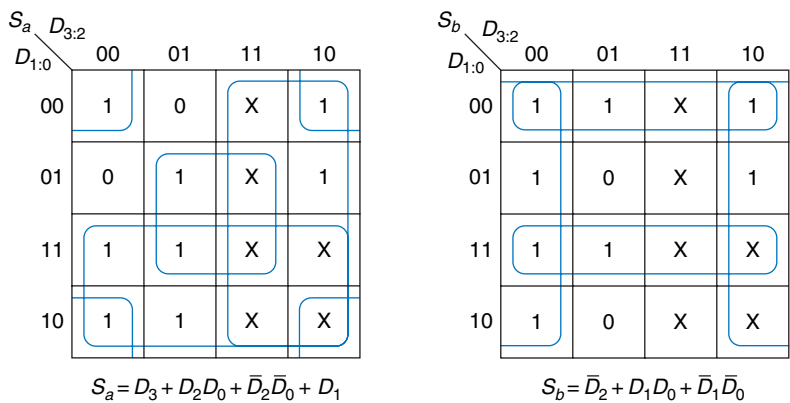


图2.53 包含无关项的K图解法

作者是否曾在现实生活中使用过卡诺图来解决实际问题？但理解卡诺图背后的原理所获得的洞察力是宝贵的。而且卡诺图经常出现在求职面试中！

2.8 组合构建模块

组合逻辑通常被组合成更大的构建模块，以构建更复杂的系统。这是抽象原则的应用，隐藏不必要的门级细节，以突出构建模块的功能。我们已经研究了三种这样的构建模块：全加器（来自第2.1节）、优先级电路（来自第2.4节）和七段显示译码器（来自第2.7节）。本节将介绍另外两种常用的构建模块：多路选择器和译码器。第5章将涵盖其他组合逻辑构建模块。

2.8.1 多路选择器

Multiplexers多路复用器是最常用的组合电路之一。它们根据select信号的值，从多个可能的输入中选择一个输出。多路复用器有时也被亲切地称为mux。

2:1 多路选择器

图2.54展示了一个2:1多路选择器的原理图和真值表，该选择器具有两个数据输入D₀和D₁、一个选择输入S以及一个输出Y。多路选择器根据选择信号在两个数据输入之间进行选择：若S = 0，则Y = D₀；若S = 1，则Y = D₁。S也被称为control signal，因为它控制着多路选择器的行为。

2:1多路选择器可以通过乘积和逻辑构建，如图2.55所示。该多路选择器的布尔方程可推导为

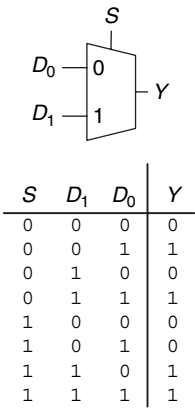


图2.54 2:1多路选择器符号和真值表

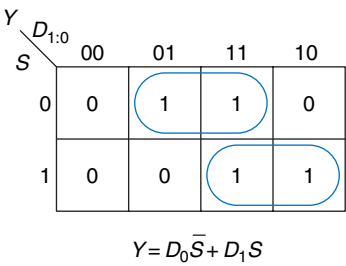


图2.55 使用两级逻辑实现的2:1多路选择器

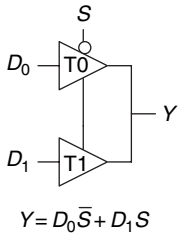
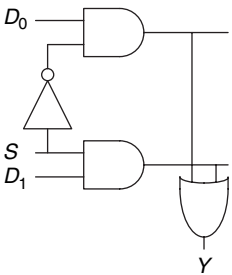


图2.56 使用三态缓冲器的多路选择器

将多个门的输出短接在一起，技术上违反了第2.1节中给出的组合电路规则。但由于在任何时刻只有一个输出被驱动，因此允许这一例外情况。

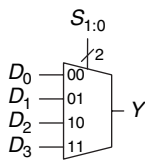


图2.57 4:1多路选择器

使用卡诺图或通过观察读出（如果 $S = 0$ 且 D_0 为1，或者如果 $S = 1$ 且 D_1 为1，则 Y 为1）。

或者，多路选择器也可以由三态缓冲器构成，如图2.56所示。三态使能端的设置确保在任何时刻恰好只有一个三态缓冲器处于激活状态。当 $S = 0$ 时，三态缓冲器T0被使能，允许 D_0 流向 Y 。当 $S = 1$ 时，三态缓冲器T1被使能，允许 D_1 流向 Y 。

更宽的多路选择器

一个4:1多路选择器有四个数据输入和一个输出，如图2.57所示。需要两个选择信号来从四个数据输入中选择一个。该4:1多路选择器可以使用积之和逻辑、三态门或多个2:1多路选择器来构建，如图2.58所示。

使三态使能的产品项可以使用与门和反相器形成，也可以使用解码器形成，我们将在第2.8.2节介绍解码器。

更宽的多路选择器，例如8:1和16:1多路选择器，可以通过扩展图2.58所示的方法来构建。通常，一个 N :1多路选择器需要 $\log_2 N$ 条选择线。同样，最佳实现方案的选择取决于目标技术。

多路复用器逻辑

多路复用器可用作lookup tables来执行逻辑功能。图2.59展示了一个用于实现双输入的4:1多路复用器。

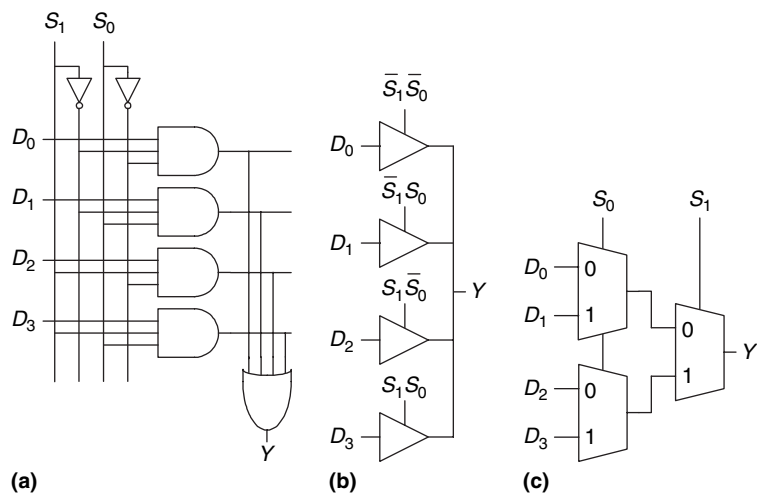


图2.58 4:1多路选择器实现: (a) 两级逻辑, (b) 三态门, (c) 层次化

与门。输入A和B作为选择线。多路复用器的数据输入根据真值表的对应行连接到0或1。通常，通过将0和1施加到适当的数据输入，一个 2^N 输入的多路复用器可以被编程以执行任何N输入的逻辑功能。实际上，通过改变数据输入，多路复用器可以被重新编程以执行不同的功能。

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

$Y = AB$

稍加巧妙设计，我们可将多路选择器的规模减半，仅用 2^{N-1} 输入的多路选择器即可实现任意N输入的逻辑函数。其策略是将一个文字量以及0和1提供给多路选择器的数据输入端。

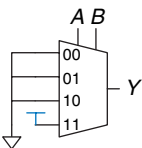


图2.59 4:1多路选择器实现两输入与函数

为说明这一原理，图2.60展示了使用2:1多路选择器实现的双输入与门和异或函数。我们从普通真值表开始，然后合并行对，通过用最右侧输入变量表示输出来消除该变量。例如，在与门的情况下，当 $A = 0$ 时， $Y = 0$ ，与B无关。当 $A = 1$ 时，若 $B = 0$ 则 $Y = 0$ ，若 $B = 1$ 则 $Y = 1$ ，因此 $Y = B$ 。随后我们根据新的、更小的真值表，将多路选择器用作查找表。

示例2.12 使用多路复用器的逻辑

Alyssa P. Hacker 需要实现函数 $Y = AB + BC + ABC$ 来完成她的毕业设计，但当她查看实验工具箱时，发现只剩下一个 8:1 多路复用器。她该如何实现这个函数？

解决方案 图2.61展示了Alyssa使用单个8:1多路复用器的实现。该多路复用器充当查找表，其中真值表中的每一行对应一个多路复用器输入。

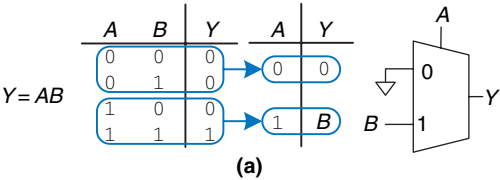


图2.60 使用变量输入的多路复用器逻辑

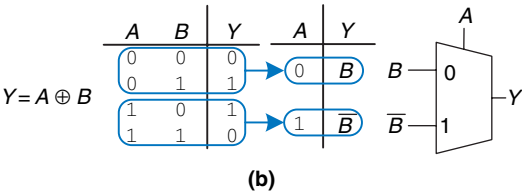
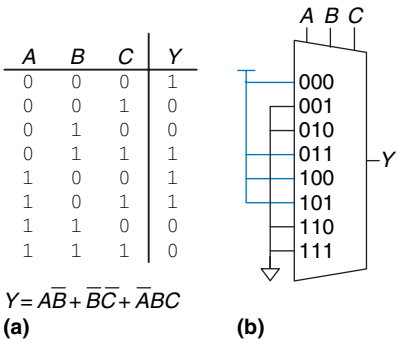


图2.61 Alyssa的电路：(a) 真值表，(b) 8:1多路复用器实现



示例2.13 使用多路复用器的逻辑，重述

Alyssa在最终演示前再次启动电路，烧毁了8:1多路复用器。（她整夜未眠后，不小心用20V电压代替了5V供电。）她向朋友们乞求备用零件，他们给了她一个4:1多路复用器和一个反相器。她能用这些零件搭建出她的电路吗？

解决方案 Alyssa 通过让输出依赖于 C，将真值表简化为四行。（她也可以选择重新排列真值表的列，让输出依赖于 A 或 B。）图 2.62 展示了新的设计。

2.8.2 解码器

一个解码器有 N 个输入和 2^N 个输出。它根据输入组合恰好断言其中一个输出。图2.63展示了一个2:4解码器。当 $A_{1:0} = 00$ 时， Y_0 为1。当 $A_{1:0} = 01$ 时， Y_1 为1。以此类推。这些输出被称为 *one-hot*，因为在任意时刻恰好有一个输出为“热”（高电平）。

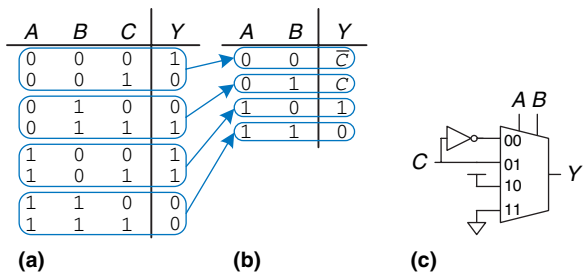


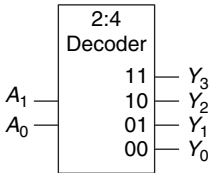
图2.62 Alyssa的新电路

示例2.14 解码器实现

使用与门和非门实现一个2:4译码器。

解决方案 图2.64展示了一种使用四个与门实现2:4译码器的方法。每个与门依赖于每个输入的原码或补码形式。通常，一个 $N:2^N$ 译码器可以由 2^N 个N输入与门构成，这些与门接受原码或补码输入的各种组合。译码器中的每个输出代表一个最小项。例如， Y_0 代表最小项 $A_1 A_0$ 。这一特性在使用译码器与其他数字构建模块时非常方便。

--



A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

图2.63 2:4译码器

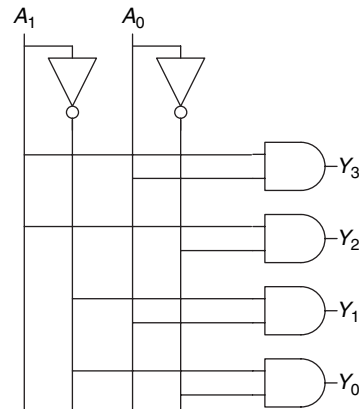


图2.64 2:4译码器实现

解码器逻辑

解码器可以与或门结合来构建逻辑函数。图2.65展示了使用2:4解码器和一个或门实现的双输入同或函数。由于解码器的每个输出代表一个最小项，该函数被构建为函数中所有最小项的或运算。在图2.65中，

$Y = \overline{AB} + AB = A \oplus B$

——

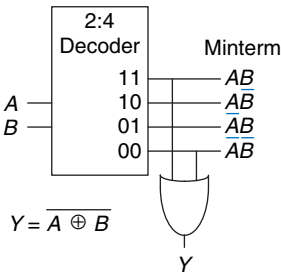


图2.65 使用解码器的逻辑函数

使用译码器构建逻辑时，最简便的方式是将函数表示为真值表或规范与或式。真值表中包含 M 个1的 N 输入函数，可通过一个 $N:2^N$ 译码器以及一个连接所有含1最小项的 M 输入或门来实现。这一概念将在5.5.6节中应用于只读存储器（ROM）的构建。

2.9 时序

在前几节中，我们主要关注电路是否能正常工作——理想情况下，使用最少的门电路。然而，正如任何经验丰富的电路设计师所证实的那样，电路设计中最具挑战性的问题之一是*timing*：让电路运行得更快。

输出响应输入变化需要时间。图2.66展示了缓冲器输入变化与后续输出变化之间的*delay*。该图称为*timing diagram*，它描绘了输入变化时缓冲器电路的*transient response*。从低电平到高电平的转换称为*rising edge*。类似地，从高电平到低电平的转换（图中未显示）称为*falling edge*。蓝色箭头表示 Y 的上升沿由 A 的上升沿引起。我们测量从输入信号50% *point*到输出信号50%点 Y 的延迟，该50%点位于信号从低电平向高电平转换时高低电平之间的中点（50%）。

当设计师谈论计算电路的*delay*时，通常指的是最坏情况下的值（传播延迟），除非上下文明确另有说明。

2.9.1 传播延迟与污染延迟

组合逻辑的特征在于其*propagation delay*和*contamination delay*。传播延迟 t_{pd} 是从任何输入发生变化到输出或输出达到最终值的最大时间。污染延迟 t_{cd} 是从任何输入发生变化到任何输出开始改变其值的最小时间。

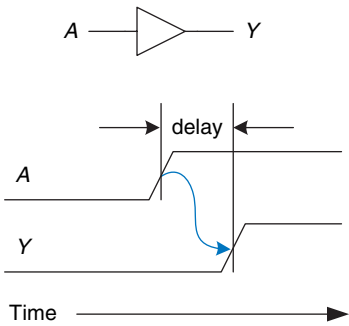


图2.66 电路延迟

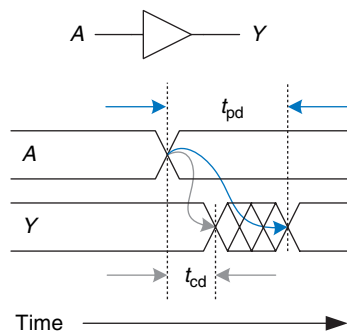


图2.67 传播延迟与污染延迟

图2.67分别用蓝色和灰色展示了缓冲器的传播延迟和污染延迟。该图显示，A初始为高电平或低电平，并在某一特定时刻切换至另一状态；我们只关心其变化这一事实，而不关心其具体数值。作为响应，Y在一段时间后发生变化。弧线表示，Y可能在A跳变后开始改变 t_{cd} ，并且Y一定会在 t_{pd} 内稳定到新值。

电路延迟的根本原因包括对电路中电容充电所需的时间以及光速。
 t_{pd} 和 t_{cd} 可能因多种原因而不同，包括

- 不同的上升和下降延迟
- 多个输入和输出，其中一些比其他的更快
- 电路在高温时变慢，在低温时变快

计算 t_{pd} 和 t_{cd} 需要深入超出本书范围的更低抽象层次。然而，制造商通常会提供数据手册，说明每个门的这些延迟。

除了已列出的因素外，传播延迟和污染延迟还取决于信号从输入到输出所经过的 *path*。图2.68显示了一个四输入逻辑电路。蓝色所示的 *critical path* 是从输入A或B到输出Y的路径。它是最长——因此也是最慢——的路径，因为输入信号需要经过

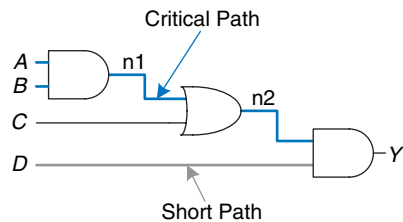


图2.68 短路径与关键路径

电路延迟通常在皮秒（1 ps = 10⁻¹²秒）到纳秒（1 ns = 10⁻⁹秒）量级。在你阅读本边栏的这段时间里，已经过去了数万亿皮秒。

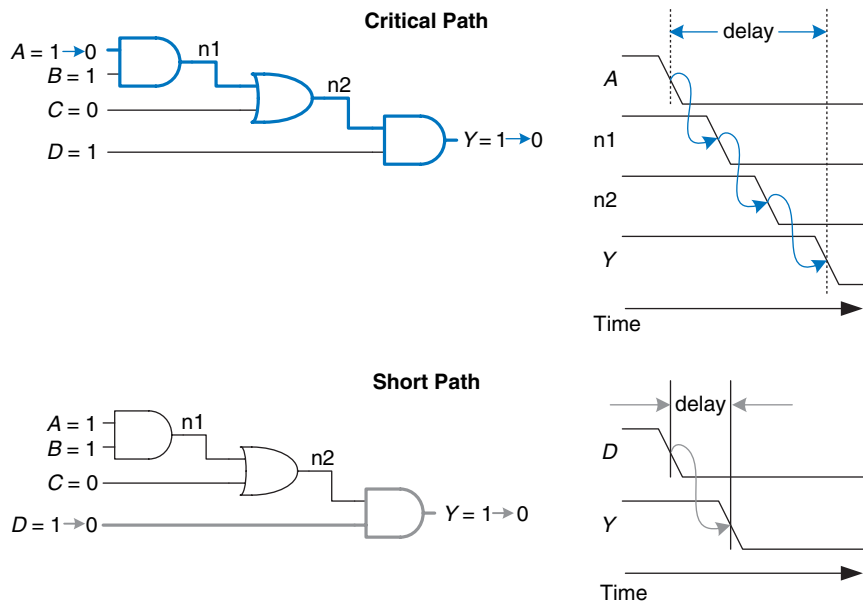


图2.69 关键路径与短路径波形

经过三个门到达输出。这条路径至关重要，因为它限制了电路运行的速度。图中灰色显示的*short path*路径从输入D到输出Y。这是通过电路的最短——因此也是最快——路径，因为输入仅经过一个门就到达输出。

组合逻辑电路的传播延迟是临界路径上每个元件传播延迟之和。污染延迟是短路径上每个元件污染延迟之和。这些延迟如图2.69所示，并由以下方程描述：

$$t_{pd} = 2t_{pd_AND} + t_{pd_OR} \tag{2.8}$$

$$t_{cd} = t_{cd_AND} \tag{2.9}$$

尽管在此分析中我们忽略了导线延迟，但数字电路现在速度极快，长导线的延迟可能与门延迟同样重要。导线中的光速延迟在附录A中讨论。

示例2.15 查找延迟

Ben Bitdiddle需要找出图2.70所示电路的传播延迟和污染延迟。根据他的数据手册，每个门的传播延迟为100皮秒（ps），污染延迟为60 ps。

解决方案 Ben 首先找出电路中的关键路径和最短路径。关键路径（在图 2.71 中以蓝色高亮显示）是从输入 A 或 B 经过三个门到达输出 Y。因此， t_{pd} 是单个门传播延迟的三倍，即 300 ps。

最短路径（图2.72中灰色所示）从输入C、D或E出发，经过两个门到达输出Y。最短路径中只有两个门，因此 t_{cd} 为120 ps。

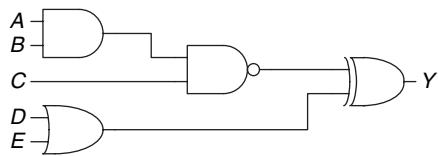


图2.70 Ben的电路

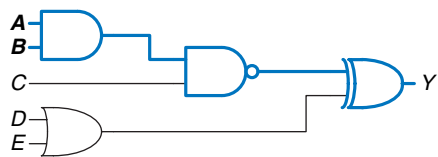


图2.71 本的关键路径

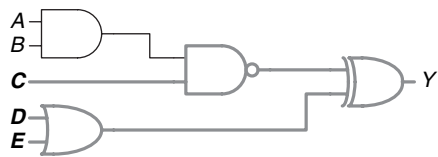


Figure 2.72 Ben's shortest path

示例2.16 多路复用器时序：控制关键型
VS. 数据关键型

比较图2.58（第83页）中三种四输入多路复用器设计的最坏情况时序。表2.7列出了各元件的传播延迟。每种设计的关键路径是什么？根据你的时序分析，为什么你会选择某一种设计而非其他？

三种设计方案各自的关键路径之一在图2.73和图2.74中以蓝色高亮显示。 t_{pd_sy} 表示从输入S到输出Y的传播延迟； t_{pd_dy} 表示从输入D到输出Y的传播延迟；该电路的 t_{pd} 是两者中的最差值： $\max(t_{pd_sy}, t_{pd_dy})$ 。

对于图2.73中的两级逻辑和三态实现，关键路径是从控制信号S到输出Y：
 $t_{pd} = t_{pd_sy}$ 。这些电路是*control critical*，因为关键路径是从控制信号到输出。控制信号的任何额外延迟都会直接增加最坏情况下的延迟。在图2.73(b)中，从D到Y的延迟仅为50 ps，相比之下，从S到Y的延迟为125 ps。

图2.74展示了使用两级2:1多路选择器实现的4:1多路选择器层次结构。关键路径是从任意D输入到输出。该电路是*data critical*，因为关键路径是从数据输入到输出： $t_{pd} = t_{pd_dyo}$

如果数据输入远早于控制输入到达，我们更倾向于选择控制到输出延迟最短的设计（图2.74中的层次化设计）。类似地，如果控制输入远早于数据输入到达，我们则更倾向于选择数据到输出延迟最短的设计（图2.73(b)中的三态设计）。

最佳选择不仅取决于通过电路的关键路径和输入到达时间，还取决于功耗、成本和部件的可用性。

表2.7 多路复用器电路元件的时序规格

Gate	t_{pd} (ps)
NOT	30
2-input AND	60
3-input AND	80
4-input OR	90
tristate (A to Y)	50
tristate (enable to Y)	35

2.9.2 毛刺

到目前为止，我们讨论了单个输入跳变导致单个输出跳变的情况。然而，单个输入跳变也可能导致*multiple*个输出跳变，这被称为*glitches*或*hazards*。尽管毛刺通常不会引发问题，但认识到它们的存在并在查看时序图时识别它们非常重要。图2.75展示了一个存在毛刺的电路及其卡诺图。

*Hazards*在第七章中，{v*} 还有另一个与微架构相关的含义，因此我们将坚持使用术语 *glitches* 来表示多输出转换，以避免混淆。

布尔方程被正确最小化，但让我们看看当A = 0、C = 1以及B从1跳变到0时会发生什么。图2.76（见第92页）展示了这一场景。短路径（以灰色显示）经过两个门，即与门和或门。关键路径（以蓝色显示）经过一个反相器和两个门，即与门和或门。

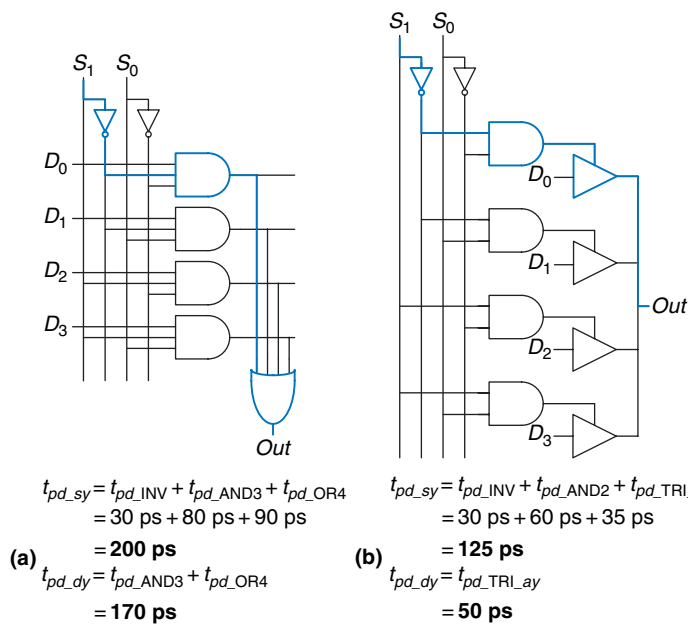


图2.73 4:1多路选择器传播延迟：(a)两级逻辑，(b)三态

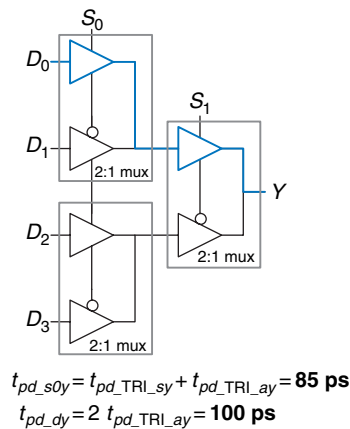


图2.74 4:1多路选择器传播延迟：使用2:1多路选择器的层次结构

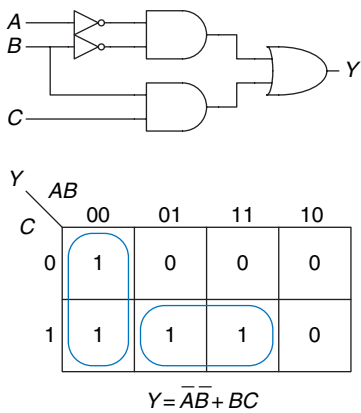


图2.75 存在毛刺的电路

当B从1变为0时，短路径上的n2在关键路径上的n1上升之前下降。在n1上升之前，或门的两个输入均为0，输出Y降至0。当n1最终上升时，Y恢复为1。如图2.76的时序图所示，Y起始为1，结束为1，但短暂地毛刺至0。

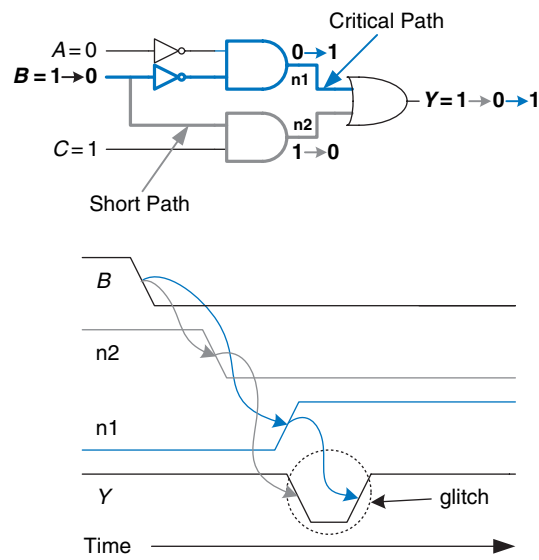


图2.76 毛刺的时序

只要我们在依赖输出之前等待传播延迟结束，毛刺就不是问题，因为输出最终会稳定到正确的结果。

如果我们选择这样做，可以通过在实现中添加另一个门来避免此故障。从卡诺图的角度最容易理解这一点。图2.77展示了输入B从ABC = 011到ABC = 001的转换如何从一个质蕴含圈移动到另一个质蕴含圈。在卡诺图中跨越两个质蕴含项边界的转换表明可能存在故障。

从图2.76的时序图可以看出，如果实现某个质蕴含项的电路在另一个质蕴含项的电路能够翻转on之前先翻转了off，就会出现毛刺。为解决此问题，我们添加另一个跨越该质蕴含项边界的圆圈covers，如图2.78所示。您可能意识到这正是一致性定理，其中添加的项AC即为一致项或冗余项。

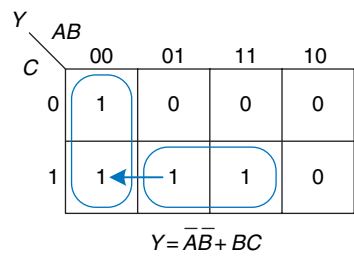


图 2.77 输入变化跨越蕴涵项边界

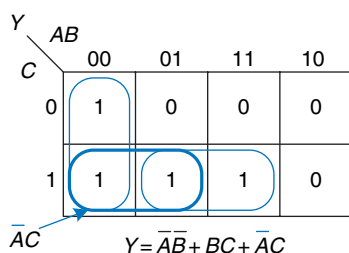


图2.78 无毛刺的K图

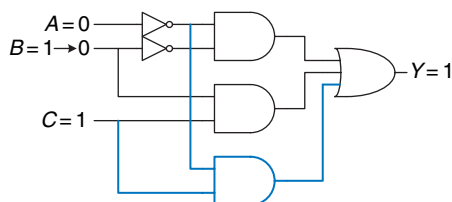


图2.79 无毛刺电路

图2.79展示了防毛刺电路。新增的与门以蓝色高亮显示。现在，当 $A = 0$ 且 $C = 1$ 时， B 上的跳变不会在输出端引起毛刺，因为蓝色与门在整个跳变过程中输出为1。

一般来说，当单个变量的变化跨越卡诺图中两个质蕴含项之间的边界时，可能会发生毛刺。我们可以通过向卡诺图中添加冗余蕴含项来覆盖这些边界，从而消除毛刺。当然，这需要以额外硬件为代价。

然而，多输入同时发生跳变也可能导致毛刺。这些毛刺无法通过增加硬件来消除。由于绝大多数有意义的系统都存在多输入同时（或近乎同时）跳变的情况，因此毛刺在大多数电路中是不可避免的现实。尽管我们已经展示了如何消除一种毛刺，但讨论毛刺的重点并非消除它们，而是意识到它们的存在。这一点在通过仿真器或示波器观察时序图时尤为重要。

2.10 总结

数字电路是一种具有离散值输入和输出的模块，其规格描述了该模块的功能和时序。本章重点讨论了组合电路，这类电路的输出仅取决于输入的当前值。

组合电路的功能可以通过真值表或布尔方程给出。任何真值表的布尔方程都可以系统地使用积之和（SOP）或和之积（POS）形式获得。在积之和形式中，函数被写为一个或多个蕴含项的和（OR）。蕴含项是文字（AND）的乘积。文字是输入变量的原变量或反变量形式。

布尔方程可以使用布尔代数规则进行简化。特别地，通过合并仅在某个文字的真值形式和互补形式上不同的蕴含项，可以将它们简化为最小和之积形式： $PA + PA = P$ 。卡诺图是一种用于简化最多四个变量函数的可视化工具。通过实践，设计者通常可以通过观察来简化少量变量的函数。对于更复杂的函数，则使用计算机辅助设计工具；此类方法和工具将在第4章中讨论。

逻辑门连接起来构成组合电路，以执行所需功能。任何以积之和形式表示的函数都可以通过两级逻辑实现：非门形成输入的反相，与门形成乘积项，或门形成和项。根据具体函数和可用的构建模块，采用多种门类型的多级逻辑实现可能更高效。例如，CMOS电路倾向于使用与非门和或非门，因为这些门可直接由CMOS晶体管构建，无需额外添加非门。使用与非门和或非门时，气泡推入法有助于跟踪反相操作。

逻辑门组合起来可以构成更大的电路，例如多路选择器、解码器和优先级电路。多路选择器根据选择输入从多个数据输入中选出一个。解码器根据输入将某个输出置为高电平。优先级电路产生一个输出，指示优先级最高的输入。这些电路都是组合逻辑构建块的例子。第5章将介绍更多的构建块，包括其他算术电路。这些构建块将在第7章中广泛用于构建微处理器。

组合电路的时序规范包括通过电路的传播延迟和污染延迟。这些延迟指示了输入变化与相应输出变化之间的最长和最短时间。计算电路的传播延迟需要识别电路中的关键路径，然后累加该路径上每个元件的传播延迟。实现复杂组合电路有许多不同的方法；这些方法在速度与成本之间提供了权衡。

下一章将转向时序电路，其输出不仅取决于当前输入值，还取决于先前输入值。换句话说，时序电路具有过去的*memory*。

练习

练习2.1 为图2.80中的每个真值表写出乘积之和标准形式的布尔方程。

(a)	(b)	(c)	(d)	(e)
A B Y	A B C Y	A B C Y	A B C D Y	A B C D Y
0 0 1	0 0 0 1	0 0 0 1	0 0 0 0 1	0 0 0 0 1
0 1 0	0 0 1 0	0 0 1 0	0 0 0 1 1	0 0 0 1 0
1 0 1	0 1 0 0	0 1 0 1	0 0 1 0 1	0 0 1 0 0
1 1 1	0 1 1 0	0 1 1 0	0 0 1 1 1	0 0 1 1 1
	1 0 0 0	1 0 0 1	0 1 0 0 0	0 1 0 0 0
	1 0 1 0	1 0 1 1	0 1 0 1 0	0 1 0 1 1
	1 1 0 0	1 1 0 0	0 1 1 0 0	0 1 1 0 1
	1 1 1 1	1 1 1 1	0 1 1 1 0	0 1 1 1 0
			1 0 0 0 1	1 0 0 0 0
			1 0 0 1 0	1 0 0 1 1
			1 0 1 0 1	1 0 1 0 1
			1 0 1 1 0	1 0 1 1 0
			1 1 0 0 0	1 1 0 0 1
			1 1 0 1 0	1 1 0 1 0
			1 1 1 0 1	1 1 1 0 0
			1 1 1 1 0	1 1 1 1 1

图2.80 习题2.1、2.3和2.41的真值表

练习2.2 为图2.81中的每个真值表写出乘积之和标准形式的布尔方程。

(a)	(b)	(c)	(d)	(e)
A B Y	A B C Y	A B C Y	A B C D Y	A B C D Y
0 0 0	0 0 0 0	0 0 0 0	0 0 0 0 1	0 0 0 0 0
0 1 1	0 0 1 1	0 0 1 1	0 0 0 1 0	0 0 0 1 0
1 0 1	0 1 0 1	0 1 0 0	0 0 1 0 1	0 0 1 0 0
1 1 1	0 1 1 1	0 1 1 0	0 0 1 1 1	0 0 1 1 1
	1 0 0 1	1 0 0 0	0 1 0 0 0	0 1 0 0 0
	1 0 1 0	1 0 1 0	0 1 0 1 0	0 1 0 1 0
	1 1 0 1	1 1 0 1	0 1 1 0 1	0 1 1 0 1
	1 1 1 0	1 1 1 1	0 1 1 1 1	0 1 1 1 1
			1 0 0 0 1	1 0 0 0 1
			1 0 0 1 0	1 0 0 1 1
			1 0 1 0 1	1 0 1 0 1
			1 0 1 1 0	1 0 1 1 1
			1 1 0 0 0	1 1 0 0 0
			1 1 0 1 0	1 1 0 1 0
			1 1 1 0 0	1 1 1 0 0
			1 1 1 1 0	1 1 1 1 0

图2.81 习题2.2和2.4的真值表

练习2.3 为图2.80中的真值表写出乘积之和规范形式的布尔方程。

练习2.4 为图2.81中的真值表写出乘积之和规范形式的布尔方程。

练习2.5 最小化练习2.1中的每个布尔方程。

练习2.6 将练习2.2中的每个布尔方程最小化。

练习2.7 为习题2.5中的每个函数设计一个相对简单的组合电路。

*Reasonably simple*意味着你不需要浪费大量门电路，但也不必花费大量时间检查每一种可能的电路实现。

练习2.8 为练习2.6中的每个函数设计一个相对简单的组合电路。

练习2.9 仅使用非门、与门和或门重复练习2.7。

练习2.10 仅使用非门、与门和或门重复练习2.8。

练习2.11 仅使用非门、与非门和或非门重复练习2.7。

练习2.12 仅使用非门、与非门和或非门重复练习2.8。

练习2.13 使用布尔定理简化以下布尔方程。通过真值表或卡诺图检查正确性。

$$\begin{aligned} & \text{(a) } Y = AC + A\bar{B}\bar{C} \quad \text{(b) } Y = AB + ABC + (A + C) \quad \text{(c)} \\ & Y \quad ABCD \quad \bar{A}BC \quad \bar{A}BC\bar{D} \quad \bar{A}BD \quad \bar{A}BCD \quad BCD \quad A \\ & = \quad \begin{array}{ccccccc} + & _ & _ & _ & + & _ & _ & + & _ & _ & + & _ & _ & \end{array} \end{aligned}$$

练习2.14 使用布尔定理简化以下布尔方程。通过真值表或卡诺图检查正确性。

$$\begin{aligned} & \text{(a) } Y = \bar{A}\bar{B}C + \bar{A}\bar{B}C\bar{D} \quad \text{(b) } Y = ABC + AB \quad \text{(c)} \\ & Y \quad ABCD \quad \bar{A}BC\bar{D} \quad A \quad B \quad C \quad D \\ & = \quad \begin{array}{ccccccc} + & _ & + & _ & + & _ & \end{array} \end{aligned}$$

练习2.15 为练习2.13中的每个函数设计一个相对简单的组合电路。

练习2.16 为习题2.14中的每个函数设计一个相对简单的组合电路草图。

练习2.17 简化以下每个布尔方程。绘制一个相对简单的组合电路来实现简化后的方程。

$$(a) Y = BC + \overline{A\overline{B}\overline{C}} + BC\overline{(b)}$$

$$Y = A + \overline{AB} + \overline{AB} + A + B =$$

$$= (c) \begin{array}{ccccccc} \overline{BCE} & \overline{ABC} & \overline{ABD} & \overline{ABE} & \overline{ACD} & (\overline{ACE} + \overline{A}) & \overline{D} & \overline{E} & \overline{BCD} \\ + & + & + & & & & & & \end{array}$$

练习2.18 简化以下每个布尔方程。绘制一个相对简单的组合电路来实现简化后的方程。

$$(a) Y = \overline{ABC} + \overline{BC} + BC (b)$$

$$Y = (A + B + C)\overline{D} + AD + B (c)$$

$$Y = ABCD + \overline{ABCD} + (B + \overline{D})\overline{E}$$

练习2.19 给出一个需要30亿到50亿行之间的真值表示例，该表可使用少于40个（但至少1个）双输入门构建。

练习2.20 给出一个具有循环路径但仍然是组合逻辑的电路示例。

练习2.21 Alyssa P. Hacker声称任何布尔函数都可以写成该函数所有质蕴含项之和的最小积和形式。Ben Bitdiddle则认为有些函数的最小方程并不包含所有质蕴含项。请解释Alyssa为何正确，或提供一个反例来证明Ben的观点。

练习2.22 使用完全归纳法证明以下定理成立。你无需证明它们的对偶形式。

(a) 幂等定理 (T3) (b) 分配律定理 (T

8) (c) 组合定理 (T10)

练习2.23 使用完全归纳法证明三个变量A、B和C的德摩根定理（T12）。

练习2.24 写出图2.82中电路的布尔方程。无需化简方程。

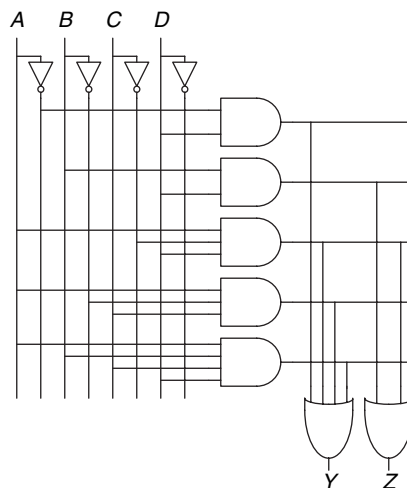


图2.82 练习2.24的电路原理图

练习2.25 最小化练习2.24中的布尔方程，并绘制具有相同功能的改进电路草图。

练习2.26 使用德摩根等效门和气泡推挤方法，重新绘制图2.83中的电路，以便通过观察找到布尔方程。写出布尔方程。

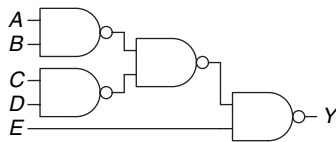


图2.83 练习2.26和2.43的电路原理图

练习2.27 对图2.84中的电路重复练习2.26。

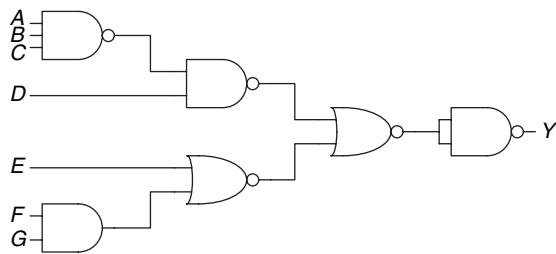


图2.84 练习2.27和2.44的电路原理图

练习2.28 找出图2.85中函数的最小布尔方程。记得利用无关项。

A	B	C	D	Y
0	0	0	0X	
0	0	0	1X	
0	0	1	0X	
0	0	1	10	
0	1	0	0	0
0	1	0	1	X
0	1	1	0	0
0	1	1	1	X
1	0	0	01	
1	0	0	10	
1	0	1	0X	
1	0	1	11	
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

图2.85 习题2.28的真值表

练习2.29 画出练习2.28中函数的电路图。

练习2.30 你的电路在练习2.29中，当某个输入发生变化时是否存在潜在的毛刺？如果没有，请解释原因。如果有，请说明如何修改电路以消除这些毛刺。

练习2.31 找出图2.86中函数的最小布尔方程。记得利用无关项。

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	X
0	0	1	1	X
0	1	0	0	0
0	1	0	1	X
0	1	1	0	X
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

图2.86 习题2.31的真值表

练习2.32 画出练习2.31中函数的电路图。

练习2.33 Ben Bitdiddle会在没有蚂蚁的晴天享受野餐。他也会在看到蜂鸟的任何一天享受野餐，以及在有蚂蚁和瓢虫的日子里享受野餐。请写出一个布尔方程，用晴天(S)、蚂蚁(A)、蜂鸟(H)和瓢虫(L)来表示他的享受(E)。

练习2.34 完成七段译码器段 S_c 至 S_g 的设计（参见示例2.10）：

(a) 假设输入大于9时输出为空白（0），推导输出 S_c 至 S_g 的布尔方程。(b) 假设输入大于9时为无关项，推导输出 S_c 至 S_g 的布尔方程。(c) 绘制(b)部分相对简单的门级实现示意图。多个输出可在适当情况下共享门电路。

练习2.35 一个电路有四个输入和两个输出。输入 $A_{3:0}$ 表示一个0到15之间的数字。如果该数字是质数（0和1不是质数，但2、3、5等是质数），则输出 P 应为TRUE。如果该数字能被3整除，则输出 D 应为TRUE。给出每个输出的简化布尔方程，并画出电路草图。

练习2.36 一个priority encoder有 2^N 个输入。它产生一个N位的二进制输出，指示输入中最高有效位为真，若无则为0。

输入为真。它还产生一个输出 $NONE$ ，当没有输入为真时该输出为真。设计一个八输入优先编码器，输入为 $A_{7:0}$ ，输出为 $Y_{2:0}$ 和 $NONE$ 。例如，如果输入是00100000，则输出 Y 应为101， $NONE$ 应为0。给出每个输出的简化布尔方程，并画出原理图。

练习2.37 设计一个修改后的优先级编码器（见练习2.36），该编码器接收一个8位输入 $A_{7:0}$ ，并产生两个3位输出 $Y_{2:0}$ 和 $Z_{2:0}$ 。 Y 表示输入中为TRUE的最高有效位。 Z 表示输入中为TRUE的次高有效位。如果没有任何输入为TRUE，则 Y 应为0。如果输入中为TRUE的位数不超过一个，则 Z 应为0。给出每个输出的简化布尔方程，并画出原理图。

练习2.38 一个 M 位的 $thermometer\ code$ ，用于表示数字 k ，由最低有效位位置上的 k 个1和所有更高有效位位置上的 $M-k$ 个0组成。一个 $binary-to-thermometer\ code\ converter$ 有 N 个输入和 2^N-1 个输出。它根据输入指定的数字生成一个 2^N-1 位的温度计码。例如，如果输入为110，则输出应为0111111。设计一个3:7二进制到温度计码转换器。给出每个输出的简化布尔方程，并画出原理图。

练习2.39 写出图2.87所示电路所实现函数的最小化布尔方程。

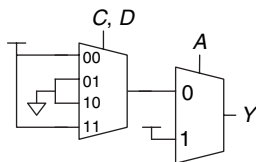


图2.87 练习2.39的多路复用器电路

练习2.40 写出图2.88中电路所执行函数的最小化布尔方程。

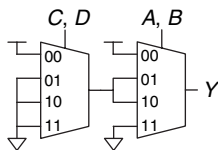


图2.88 练习2.40的多路复用器电路

练习2.41 使用图2.80(b)中的函数实现

- (a) 一个8:1多路选择器 (b) 一个4:1多路选择器和一个反相器 (c) 一个2:1多路选择器和两个其他逻辑门

练习2.42 使用以下方法实现练习2.17(a)中的函数

- (a) 一个8:1多路复用器
(b) 一个4:1多路复用器，且无其他门电路
(c) 一个2:1多路选择器、一个或门和一个反相器

练习2.43 确定图2.83中电路的传播延迟和污染延迟。使用表2.8中给出的门延迟。

练习2.44 确定图2.84中电路的传播延迟和污染延迟。使用表2.8中给出的门延迟。

表2.8 习题2.43–2.45和2.47–2.48的门延迟

Gate	t_{pd} (ps)	t_{cd} (ps)
NOT	15	10
2-input NAND	20	15
3-input NAND	30	25
2-input NOR	30	25
3-input NOR	45	35
2-input AND	30	25
3-input AND	40	30
2-input OR	40	30
3-input OR	55	45
2-input XOR	60	40

练习2.45 绘制一个快速3:8译码器的原理图。假设门延迟如表2.8所示（且仅可使用该表中的门）。设计你的译码器使其具有尽可能短的关键路径，并指出该路径是什么。其传播延迟和污染延迟分别是多少？

练习2.46 设计一个从数据输入到输出具有最短可能延迟的8:1多路复用器。你可以使用第90页表2.7中的任何门电路。画出原理图。利用表中的门延迟，确定该延迟。

练习2.47 重新设计练习2.35中的电路，使其尽可能快。仅使用表2.8中的门电路。画出新电路并标出关键路径。其传播延迟和污染延迟是多少？

练习2.48 重新设计练习2.36中的优先级编码器，使其尽可能快。你可以使用表2.8中的任何门。画出新电路并标出关键路径。其传播延迟和污染延迟是多少？

练习2.49 另一种思考晶体管级设计的方法（见第1.7节）是利用德摩根定理来考虑上拉网络和下拉网络。直接根据以下方程设计晶体管级门电路的下拉网络。然后，对方程应用德摩根定理，并利用重写后的方程绘制上拉网络。同时，说明所使用的晶体管数量。不要忘记绘制（并计数）用于对输入取反的反相器（如果需要）。

$$(a) \quad W = \overline{A + BC + \overline{CD}}$$

$$(b) \quad X = \overline{\overline{A}(B + C + D) + A\overline{D}}$$

$$(c) \quad Y = \overline{\overline{A}(BC + \overline{B}\overline{C}) + A\overline{B}\overline{C}}$$

练习2.50 对以下方程重复练习2.49。

$$(a) \quad W = \overline{(A + B)(C + D)}$$

$$(b) \quad X = \overline{\overline{A}B(C + D) + A\overline{D}}$$

$$(c) \quad Y = \overline{\overline{A}(B + \overline{C}D) + A\overline{B}CD}$$

面试问题

以下练习题展示了数字设计岗位面试中曾出现的问题。

问题2.1 仅使用与非门绘制一个两输入异或函数的原理图。最少能用多少个？

问题2.2 设计一个电路，用于判断给定月份是否有31天。月份由4位输入 $A_{3:0}$ 指定。例如，若输入为0001，则月份为一月；若输入为1100，则月份为十二月。电路输出Y仅在指定月份有31天时为高电平。写出简化后的方程，并使用最少门数绘制电路图。（提示：记得利用无关项。）

问题2.3 什么是三态缓冲器？它如何以及为何被使用？

问题2.4 如果一个门或一组门能够用于构造任意布尔函数，则称其为通用的。例如，集合{AND, OR, NOT}是通用的。

- (a) AND门本身是通用的吗？为什么是或为什么不是？
- (b) 集合{OR, NOT}是通用的吗？为什么是或为什么不是？
- (c) NAND门本身是通用的吗？为什么是或为什么不是？

问题2.5 解释为什么电路的污染延迟可能小于（而不是等于）其传播延迟。

